Mirage: Provable Defense Against LLM KV Cache Timing Side Channels with Topology-Aware Isolation

Xiaobei Yan
Nanyang Technological University
xiaobei002@e.ntu.edu.sg

Tianwei Zhang
Nanyang Technological University
tianwei.zhang@ntu.edu.sg

arXiv:2611.10756v1 [cs.CR] 7 Nov 2026

***Abstract*—Large Language Models (LLMs) are increasingly deployed as shared multi-tenant services, relying on advanced optimizations like KV cache and prefix sharing to improve throughput and reduce latency. However, these optimizations introduce a critical vulnerability: attackers can exploit time-to-first-token (TTFT) variations between KV cache hits and misses to infer sensitive prompt tokens via prefix-matching side-channel attacks. Existing defenses indiscriminately restrict cross-tenant cache sharing, which incurs prohibitive performance overhead and negates the fundamental efficiency benefits of the KV cache.**

We propose Mirage, a topology-aware emission obfuscation framework that provably bounds timing leakage without sacrificing the performance gains of cache sharing. Instead of stalling the GPU or forcing redundant recomputations, Mirage delays token emission at the gateway so the attacker observes a natural cache miss distribution, while the GPU seamlessly continues serving other requests at full speed. Mirage uses a Privacy Odometer that persistently tracks the topological novelty of prefix fan-out expansions to distinguish aggressive probing from benign traffic. Once a prefix exceeds a safety threshold, Mirage engages a two-phase policy that uses a dynamic estimator to pad the TTFT to a calibrated cache-miss baseline. On a 500-sample public corpus across three models, Mirage suppresses the timing side-channel to near the random-guessing floor, while maintaining a significantly lower TTFT than defense baselines. We further show that Mirage withstands a strong adaptive adversary and the same principle carries over to semantic caching layers.

I. Introduction

Large language models (LLMs) are increasingly deployed as shared cloud infrastructure, driving their adoption in privacy-sensitive fields like medical [1], finance [2], and coding assistants [3]. This paradigm shift makes inference latency a first-order system constraint. Users expect interactive responsiveness, while providers must efficiently amortize scarce GPU compute and memory across concurrent requests. While early optimizations focused on weight quantization [4], [5], [6], model compression [7], and backend enhancements [8], modern serving engines increasingly rely on key-value (KV) cache management and prefix reuse to eliminate redundant computations. By organizing cached prefixes through block hashes or radix-tree structures, systems like vLLM and SGLang allow subsequent requests sharing initial tokens to bypass the expensive prefill phase and directly reuse cached KV tensors [9], [10]. Today, prefix sharing serves as a foundational performance primitive across modern APIs, significantly accelerating workloads that rely on long prompts, repeated templates, retrieval-augmented generation (RAG), and multi-turn conversations [11], [12], [13], [14], [15].

Unfortunately, the KV cache sharing scheme exposes a timing oracle. A request whose prefix hits the shared KV cache reaches the first generated token much faster than a request whose prefix must be computed from scratch. Prior work has shown that an attacker can exploit this time-to-first-token (TTFT) gap using only ordinary API access: the attacker sends many guessing probes, measures whether the probe behaves like a cache hit, and repeats the process to recover private prompts, system instructions, or fields from another user's input [16], [17], [18], [19]. This risk is especially acute in multi-tenant deployments because sensitive content such as medical questions, legal facts, enterprise documents, and proprietary agent prompts may coexist with performance-oriented cross-user cache sharing.

Existing mitigations leave an uncomfortable trade-off. Per-user cache partitioning and cache salts prevent cross-tenant cache hits, but also remove the cross-tenant reuse that prefix caching was designed to provide [20], [21]. Selective-sharing systems such as SafeKV [22] reduce this cost by classifying which cache entries are sensitive, but they still respond by *isolating* all entries classified as sensitive. Therefore, a genuine user of a sensitive prefix loses all cache reuse, the very benefit caching provides. Besides, their security relies entirely on an imperfect classifier: a false negative leaks the sensitive prefix completely, while a false positive needlessly forfeits cache reuse for benign traffic. Fundamentally, these defenses label content while ignoring attackers' *behaviors*, so they cannot distinguish a benign user reading a sensitive document from an adversary hammering it with candidate probes.

A seemingly simpler alternative is timing obfuscation: add delay or jitter to suspicious first-token emissions. However, this introduces several unique challenges: First, indiscriminately adding noise negates the latency-reduction benefits of the KV cache. As pointed out in prior studies [22], completely masking the timing channel requires padding all cache-hit response time to the upper tail of the cache-miss distribution. This directly obliterates the substantial latency reductions (often up to 73%) typically provided by prefix caching, rendering the underlying performance optimization practically meaningless. Second, because natural cache miss latencies fluctuate dynamically with server load, sampling an artificial delay that convincingly mimics the real distribution is non-trivial. Third, obfuscation

must be applied with care in streaming APIs: if only the first token is held back while the engine keeps decoding, the buffered tokens are released in a burst, so a correct implementation shifts the entire emission stream rather than its first token alone. Furthermore, establishing rigorous mathematical bounds on an attacker's residual advantage when facing such obfuscation remains an open and necessary challenge. Finally, designing this defense to integrate easily into production-grade serving environments as a plug-and-play module with minimal revision to current frameworks is a critical practical consideration.

To address these challenges, this paper presents Mirage, a theoretically bounded defense that formally limits residual timing leakage while preserving the compute savings of prefix caching. Mirage is based on a simple observation: cache probing attacks are not arbitrary traffic: the attacker must target a sensitive prefix. Because the unknown private text possesses high information entropy, while each timing probe reveals at most a few bits (the hit/miss verdict), the attacker is mathematically forced to issue a massive number of distinct candidate guessing probes beneath that exact same prefix. In the prefix cache's logical topology, this creates a distinctive high-fan-out pattern: many novel outgoing branches stemming from a single metered node. Mirage turns this fundamental attack requirement into a control-plane budget. It tracks how many topologically novel continuations a sensitive prefix has accumulated, decays this state over time, and moves the prefix into an emission lockdown phase once the safety budget is exhausted. Because this signal sits on the critical path of every request, it is kept deliberately lightweight, and Mirage shows that a single, cheap behavioral signal is sufficient to close the channel. When a request enters lockdown, Mirage does not stop the GPU. Instead, it performs *time shifting* at the API emission layer. After the first token is produced, Mirage samples a target latency from a calibrated cache-miss distribution and computes a constant offset between the true token arrival time and the target emission time. The gateway then applies this same offset to every chunk in the stream. As a result, the attacker observes a natural miss-like TTFT and natural inter-token latency (ITL). The GPU remains free to continue serving other work at full speed, while the client-visible trace is shifted as a whole.

We implement Mirage in SGLang and evaluate it against three representative KV-cache timing attacks across Qwen, DeepSeek, and Llama-family models. Mirage reduces the success rate of all three attack baselines to near the random-guessing floor while preserving benign public-prefix latency. By contrast, isolation-style defenses forfeit shared-prefix reuse and inflate benign median TTFT by roughly 45% on the 14B model at moderate load, and several-fold on smaller models as load rises.

In summary, this paper makes the following contributions:

- **Metering fan-out rather than classifying secrets.** Any timing attacker recovering a high-entropy secret must open many novel branches beneath the same prefix; we prove a Fano-style lower bound on this fan-out and make it the primary detection signal. A coarse content-sensitivity weight only scales how fast a prefix accrues risk, so Mirage acts on the attacker's unavoidable exploration behavior instead of deciding per request whether a prompt contains a secret.
- **Isolation-level security without sacrificing cache reuse.** Mirage serves all prefixes from the shared KV cache at full GPU speed, applying stream-level time-shifting at the gateway only during lockdown. By eliminating the redundant recomputations forced by isolation-based solutions, Mirage retains up to $1.6\times$ higher serving throughput, maintains near-zero memory overhead, and confines latency penalties strictly to sensitive traffic that is under active probing, all with provable per-request latency and capacity bounds (Appendix B).
- **Stream-level timing equalization with provable bounds.** We theoretically bound the attacker's residual advantage by an equalization gap δ_{eq} and design a dynamic tail-quantile clamp that shifts the entire token stream. We empirically show this eliminates whole-trace timing leaks, keeping state-of-the-art multivariate and ITL-aware distinguishers at chance across three model scales and batched loads.
- **A plug-and-play realization.** Implemented as a module over unmodified SGLang, one Mirage policy drives three published timing attacks to the guessing floor on three model families, resists a white-box adaptive adversary, and carries over to semantic caches.

II. Background

A. LLM Inference and Streaming Latency

Transformer-based LLMs generate text autoregressively using self-attention over query, key, and value vectors [23]. Serving a request has two stages. In the *prefill* stage, the model processes the entire input prompt in parallel, computes the per-layer key/value tensors for all prompt tokens, and produces the first output token. In the *decode* stage, the model generates subsequent tokens one at a time; each step computes the key/value tensors for the newest token and attends over the previous context. Modern serving systems treat these stages differently because prefill is compute-heavy and prompt-length dependent, while decode is latency-sensitive and sequential [24].

Interactive LLM APIs commonly stream tokens as they are generated. Two latency metrics are therefore visible to clients. TTFT is the elapsed time from request submission to the first emitted token, and mostly captures queueing plus prefill work. *Inter-token latency* (ITL), also called time per output token, is the gap between consecutive emitted tokens and mostly captures decode cadence. For a completion of N generated tokens, user-visible latency can be decomposed as $\mathrm{TTFT} + \sum_{i=2}^{N} \mathrm{ITL}_i$.

B. KV Cache and Prefix Reuse

The KV cache stores the key and value tensors computed for previous tokens so that later attention steps can reuse them instead of recomputing the entire context. This reduces the per-step decoding cost from repeatedly processing all prior tokens to processing only the newly generated token, which is essential

for long contexts and multi-turn conversations [25]. However, KV tensors are large and must be managed carefully across batches, active requests, and memory tiers. PagedAttention in vLLMs, for example, partitions KV memory into blocks and uses metadata to support flexible allocation and reuse [9].

Prefix caching extends KV reuse across requests. Because causal attention makes a token's KV tensors depend on all preceding tokens, two requests can share cached tensors only for an identical prefix starting at the first token. If one request begins with the same token sequence as an earlier request, the serving engine can reuse the cached prefix and prefill only the unmatched suffix. This optimization is widely adopted in open-source systems and commercial APIs because many workloads repeatedly use system prompts, few-shot examples, RAG templates, long documents, or conversation history [10], [11], [12], [13], [14], [15].

C. Timing-based Prefix-Probing Attacks

To facilitate prefix sharing, serving systems employ dedicated index structures to map input tokens to their corresponding KV tensors. For example, SGLang utilizes a Radix Tree (RadixAttention) to manage cached prefixes [10], while vLLM relies on block-level hashing mechanisms [9]. Though structurally different, both designs yield the same operational outcome: an incoming request either matches a cached prefix to bypass redundant computations, or misses and triggers a full prompt recomputation. Crucially, these cache entries reside in the volatile data plane (e.g., GPU memory) and are managed by heuristic eviction policies (like LRU) optimizing for performance rather than privacy.

This architectural choice inadvertently exposes a highly observable timing oracle [18], [16], [17]. Because a prefix cache hit removes intensive prefill computation from the critical path, the TTFT drops significantly compared to a cache miss. In a multi-tenant environment, an attacker can exploit this discrepancy to reverse-engineer co-tenants' private data. Specifically, for a targeted prefix P, the attacker issues probes of the form $P\|c_i\|R$, where c_i is a candidate token or phrase and R is an optional suffix that keeps the request syntactically valid. If $P\|c_i$ matches a victim-cached continuation, the backend can reuse a longer cached prefix and the attacker's TTFT is drawn from a faster hit-like distribution; otherwise, the backend recomputes more of the prompt and the TTFT is drawn from a slower miss-like distribution. Repeating this test over many candidates [18], [17], [16] recovers the secret one token or span at a time.

III. Threat Model and Motivation

A. Threat Model

We consider a multi-tenant LLM serving backend that enables cross-request prefix caching for performance: the backend may reuse KV tensors when an incoming prompt shares a token prefix with a previous request. This is the setting studied by recent KV-cache leakage attacks and defenses [18], [17], [16], [20], [22]. We assume a well-functioning and secure operating system or hypervisor, and that standard transport security and authentication are in place; the leakage considered here arises solely from API-visible timing, not from plaintext network exposure.

Secrets and Attacker Objective. The victim's secret is a token sequence that should not be inferable by another tenant, such as a system prompt or a span of a private document. We focus on the incremental reconstruction problem used by prior attacks: given a known or guessed prefix P, the attacker tries to identify the next secret token or short span that follows P. The attacker may know public prompt templates, tokenizer behavior, or domain context, but not the victim's exact private continuation. A successful attack can leverage timing observations to distinguish correct continuation from candidate alternatives with probability significantly above guessing.

Adversary Capabilities. The attacker is an ordinary black-box API client. He can create one or more accounts, issue chosen prompts, replay prompts, send probes concurrently or in carefully timed batches, and adapt later probes based on earlier timing measurements. He can observe the full streaming trace returned to his own requests: request submission time, TTFT, and the inter-token gaps of subsequent chunks. We grant the whole trace rather than TTFT alone so the evaluation also accounts for any residual signal in the post-TTFT emissions.

We also allow the attacker to directly flush the cache or create cache pressure by sending unrelated dummy requests, which triggers ordinary LRU-style eviction in the GPU-resident KV cache. Furthermore, the attacker possesses domain knowledge and can fine-tune his local auxiliary models to optimize candidate generation and maximize attack efficiency. However, the attacker has no privileged server access: he cannot read other users' prompts, inspect KV tensors, tamper with the control plane, etc.

Defense Goals. Our goal is to bound the number of informative novel probes available for a sensitive prefix and, once that budget is exhausted, make the client-visible emission trace independent of whether the probe hits a victim's cached prefix. The protected observation is the whole streaming trace, not only TTFT: after lockdown, both the first-token emission time and subsequent inter-token latencies should be consistent with a natural miss-like execution.

Our solution does not claim to hide all information in the first grace-period probes; instead, it sets the topology budget below the number of novel candidates needed for reliable extraction. It also does not address attacks that require privileged access, direct KV cache content or model-output leakage, semantic leakage unrelated to prefix-cache timing, token-count leakage, or hardware and microarchitectural side channels outside the API timing surface. Public templates and explicitly trusted sharing groups are considered safe cache-sharing domains and may bypass our monitoring policy.

B. Motivation

Three observations about the timing side channel motivate our design. First, the channel is narrow but the secret is wide, and this asymmetry makes the attacker's *behavior* distinguishable. A single probe reveals at most a few bits (the hit/miss

verdict plus a coarse magnitude signal), while the secrets worth stealing carry far more entropy than that. Whatever candidate strategy the attacker adopts, it must therefore accumulate its answer across many distinct continuations under the same prefix, a sustained fan-out that no benign user of that prefix reproduces. The attack is thus visible in the shape of the traffic itself, before any judgment about which content is secret, and a defense can key on this signal instead of on content labels.

Second, the attacker observes latency, not computation under our threat model, which admits a cheaper equivalent of isolation. Existing defenses close the channel by actually withholding reuse: a protected prefix is recomputed from scratch for each trust boundary, which works against the very purpose of the KV cache. But since the client sees only timing, the same view can be produced without the recomputation: keeping serving the prefix from the shared cache in the backend, and delaying the emission so the response looks like a miss in the frontend. If the added delay follows the same distribution as a genuine miss, the two constructions are indistinguishable from the attacker's viewpoint, yet the GPU time that recomputation would burn is freed for other pending requests. In the worst case, where every request of a prefix ends up padded, this degrades to the same latency as isolation; in every other case, the reclaimed compute is pure gain.

Third, a defense is only deployable if it respects the serving stack it protects. Inference engines evolve quickly, so the defense should not modify the engine's scheduler, cache structures, or kernels; and since its logic runs on the critical path of every request in a latency-sensitive service, that logic must itself be near-free. This pushes the design toward a self-contained control-plane layer: light bookkeeping and emission control at the gateway, no GPU work, and no modification to the underlying serving engine.

IV. Methodology

A. Design Overview

Mirage is a stream-level defense for shared-prefix serving. It does not prevent prefix caching or stall the GPU when a suspicious request appears; instead, it separates privacy state tracking from physical execution and controls only the emitted timing. Three components work together: a *behavioral* detector (the Privacy Odometer) that meters the attacker's unavoidable fan-out instead of guessing which content is secret; a *decoupled* emitter (the gateway time-shift) that hides the hit/miss bit by shifting the whole client-visible stream while the GPU keeps running; and a *calibrated* target that drives a padded hit toward statistical indistinguishability from a genuine miss, up to a small residual we measure (δ_{eq}).

As illustrated in Figure 1, the architecture is a decoupled defense layer appended to an existing LLM engine and joined at a gateway: the backend GPU continues to process tokens at full speed, while the gateway independently shapes the client-visible timing trace. The workflow spans two lanes joined at a gateway. First, incoming client requests enter the unmodified *Existing engine* (the data plane), comprising the *Prefix Cache* and *GPU execution* core. The request is evaluated against the native radix tree to maximize cache hits and skip redundant prefill computations. In parallel, the request is evaluated by Mirage's decoupled *control plane*. The *Privacy Odometer* meters the topological novelty of the request's continuation beneath the sensitive prefix. It updates a time-decayed risk score S_P and issues a Phase Flag (Phase 0, 1, or 2) dictating the defense posture. Operating alongside it, the *Miss Estimator* maintains a dual-track exponential moving average (EMA) using actual TTFT feedback (x_t) from the GPU, continuously sampling a dynamic, load-aware cache-miss target latency (τ). Finally, the *Gateway* intercepts the raw tokens and acts as the decisive traffic router. Based on the Phase Flag from the odometer, it splits emissions into two diverging paths:

- **Benign/Grace Stream (Phase 0/1):** Public traffic (i.e., general-purpose queries, standard instruction templates, or any request assigned a zero sensitivity weight) alongside the initial grace-period queries of sensitive prefixes, simply pass through ($e_i = a_i$) with unaltered fast TTFTs, retaining 100% of prefix cache's performance benefits.
- **Protected Stream (Phase 2):** Once the probing budget is exhausted, the gateway triggers an emission lockdown. It computes a constant offset (Δ_{pad}) bridging the actual execution time and the dynamic miss target (τ). The gateway then shifts every streamed chunk by this identical offset ($e_i = a_i + \Delta_{\mathsf{pad}}$). As a result, the attacker observes a natural miss-like TTFT while the subsequent inter-token latencies remain uncompressed and natural.

The architectural separation between the defense layer and native engine is specifically designed for robust security. The underlying GPU may evict KV blocks under memory pressure, but the Mirage odometer remains securely in CPU memory or a distributed store, decaying only according to its own logical policy. This completely decouples the privacy state from the physical cache, preventing an attacker from using cache-flush floods to reset the defense and launch subsequent probes.

B. Privacy Odometer

The Privacy Odometer is a control-plane counter that tracks the *logical topology* of prefix reuse. Instead of metering physical cache hits (which would allow attackers to test incorrect candidates for free until they succeed), the odometer increments a risk score whenever a request creates a previously unseen continuation (a "topologically novel branch") beneath a metered prefix. Exact retries open no new branches, so they are never penalized.

For a request r_j arriving at time t_j, Mirage decomposes the prompt into a metered prefix P and a continuation branch b_j. The prefix P acts as a fixed metering anchor that does not move as the prompt grows. The branch b_j is a cryptographic hash of the *entire* continuation that follows P. This full-continuation hashing ensures that any attempt to explore different candidates is robustly registered as a novel branch, even if injected into the middle of a template ($P \| \mathsf{candidate} \| \mathsf{suffix}$). Genuine retries naturally hash to the same branch.

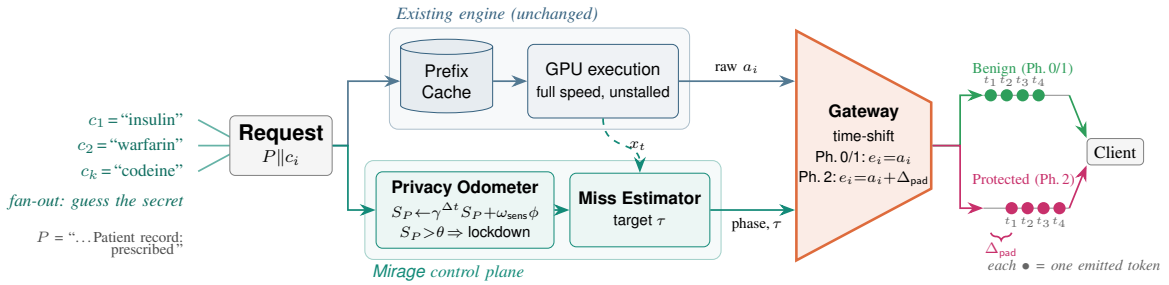

Fig. 1. An attacker appends candidate guesses c_i to a sensitive prefix P and times the reply: a cache hit reveals the match. Mirage's control plane (teal) meters this fan-out while the unchanged engine (blue) serves the shared cache at full speed; the gateway (coral) passes benign streams through and shifts every token of a locked-down stream by one offset Δ_{pad} (rose), so timing no longer reveals the hit.

Let B_P be the persistent set of branches already observed below P. The topological novelty term is defined as:

$$\phi_{\text{topo}}(r_j) = \begin{cases} 1, & b_j \notin B_P, \\ 0, & b_j \in B_P. \end{cases} \tag{1}$$

Trust Scope as an Accounting Key. Before calculating the risk score, Mirage assigns the request to a *Trust Scope (c)*. A scope acts strictly as a logical *accounting key* (a behavioral risk ledger) rather than a physical cache boundary. In practice, the system maintains a separate risk ledger for each uniquely authenticated user. For example, if Dr. A and Dr. B both query the same sensitive patient record, Mirage serves both requests using the exact same physical KV cache in the GPU to maximize performance. However, their distinct probing behaviors are metered in completely separate scope ledgers. Consequently, the natural diversity of queries from many benign doctors legitimately sharing one record can never pool together into a single false alarm.

To defend against Sybil attacks, unauthenticated or anonymous traffic is collapsed into a single pooled scope. An adversary cannot bypass the mechanism by generating thousands of fake connections or rotating IP addresses to acquire a fresh grace budget for each probe, as all their anonymous attempts will accumulate against the identical pooled ledger and swiftly trigger the defense.

Risk Score Update. After computing ϕ_{topo}, Mirage inserts b_j into B_P and updates the risk score for the specific (P, c) pair:

$$S_{P,c}(t_j) = \min\Big\{S_{\max},\ \gamma^{t_j - t_{j-1}} S_{P,c}(t_{j-1}) + \omega_{\text{sens}}(P)\big(\phi_{\text{topo}}(r_j) + \rho(1 - \phi_{\text{topo}}(r_j))\big)\Big\}. \tag{2}$$

Here, $\gamma \in (0, 1]$ is the temporal decay factor, $\omega_{\text{sens}}(P)$ is the prefix sensitivity weight, and $\rho \in [0, 1]$ is an optional penalty for exact retries (we default to $\rho = 0$).

The score is capped at $S_{\max}$. This cap prevents Denial-of-Service (DoS) attacks where a malicious user floods a shared prefix to lock it permanently. According to Equation 20 in Section V, a locked prefix is mathematically guaranteed to decay below the threshold within a bounded idle time. Even if an attacker intentionally floods a pooled scope to sustain a lock, the result is merely a temporary latency degradation. This degraded latency exactly matches the baseline penalty that strict isolation defenses (e.g., SafeKV, per-user partitioning, vLLM salt-scoped caching) impose unconditionally on *every* cross-boundary access. In other words, such flooding degrades system availability for that prefix, but never breaches confidentiality.

Finally, this privacy state (the decayed score and compact branch set B_P) is deliberately decoupled from the GPU data plane. As the odometer lives securely in the control plane memory, attackers cannot reset their risk scores by intentionally triggering GPU LRU cache evictions or flushes.

C. Sensitivity Gate and Gateway Multiplexer

The gateway first evaluates a sensitivity weight $\omega_{\text{sens}}(P) \in [0, 1]$ for the metered prefix. In Mirage, a "sensitive token" is not meant to be an exhaustively enumerated secret vocabulary. It is any token, structured span, application route, or trust-boundary label indicating that continuations below the current prefix may encode private information. The weight is therefore a policy interface: $\omega_{\text{sens}}(P) = 0$ means public (i.e., general-purpose queries, standard instruction templates, or any request assigned a zero sensitivity weight) or trusted-sharing traffic, while $\omega_{\text{sens}}(P) > 0$ means the prefix is privacy-bearing and should be metered.

Our prototype instantiates this interface with a reproducible lexical and structural gate for the medical-consultation benchmark. It assigns high weight to protected domain contexts (`patient`, `clinical`, `healthcare`), names and surnames from public Census-style lists, medical conditions and medications, legal/financial terms, and structured identifiers such as email addresses, phone numbers, long string/numeric identifiers, or private-location contexts. Crucially, the security of Mirage does *not* rest on this list being complete. The safe deployment posture is *fail-closed*: a provider sets $\omega_{\text{sens}} = 1$ by default for an entire endpoint, tenant namespace, private document class, or system-prompt template, so every request on a private surface is metered without content inspection, and the lexical gate serves only as a whitelist that lowers ω_{sens} for known-public templates to reclaim their cache speed. A secret that "slips through" the list is therefore still protected by the fan-out budget of its metered endpoint; content scanning is a performance optimization, never the foundation of the guarantee. Nor does fail-closed *metering* mean fail-closed *padding*: a metered request is watched, not slowed. Padding engages only once a scope's fan-out budget is exhausted, so benign traffic keeps the fast cache path while the behavioral signal decides when, and for whom, emission is equalized.

Mirage then uses a threshold θ to switch from a grace period to lockdown:

$$\mathsf{phase}(P,t) = \begin{cases} \mathsf{Phase\ 0}, & \omega_{\mathsf{sens}}(P) = 0, \\ \mathsf{Phase\ 1}, & \omega_{\mathsf{sens}}(P) > 0 \text{ and } S_P(t) \le \theta, \\ \mathsf{Phase\ 2}, & S_P(t) > \theta. \end{cases} \tag{3}$$

Phase 0 is a performance whitelist path: requests identified as non-sensitive by the application layer or sensitivity gate (e.g., standard public instruction templates, system prompts, or general-knowledge queries) bypass the odometer and receive no emission padding, so they retain the native fast-cache TTFT. Phase 1 is the grace period for metered prefixes; the request also passes through unpadded because the prefix has not yet consumed its topology budget. Neither phase adds any equalization delay; the only residual cost is a small, constant control-plane term (the sensitivity classification and gateway interception applied to every request, quantified in Section VI-C), not a per-request padding cost. Phase 2 is lockdown: emissions for that prefix are shaped to a miss-like distribution to shut down the timing oracle channel. Since every novel branch increases the score by at least $\omega_{\mathsf{sens}}(P)$ when $\rho = 0$, a rapid attacker can create at most

$$K_{\max}(P) = \left\lfloor \frac{\theta}{\omega_{\mathsf{sens}}(P)} \right\rfloor \tag{4}$$

distinct Phase-1 candidate branches before the prefix enters lockdown and every later probe is padded.

Configurable Grace Budget. Let G be the number of unpadded novel branches a sensitive prefix is allowed before lockdown. The budget θ takes the form $G\,\omega_{\mathsf{sens}} \le \theta < (G+1)\omega_{\mathsf{sens}}$, as each novel branch adds exactly ω_{sens} to the score (Equation 2) with $\rho = 0$, so after the G-th distinct candidate the score is $G\,\omega_{\mathsf{sens}}$. The left inequality ($\theta \ge G\,\omega_{\mathsf{sens}}$) keeps those first G probes in Phase 1, and the right inequality ($\theta < (G+1)\omega_{\mathsf{sens}}$) forces the $(G{+}1)$-th probe, which lifts the score to $(G{+}1)\omega_{\mathsf{sens}}$, over the threshold and into lockdown. The two bounds together pin lockdown to exactly the $(G{+}1)$-th novel candidate. Our main configuration adopts $G = 1$ by setting $\theta = 1.0$ with $\omega_{\mathsf{sens}} = 1.0$: the *first* novel branch beneath a sensitive prefix evaluates to $S_P = 1.0 \le \theta$ and passes through Phase 1 with zero overhead, so the first genuine visitor of a sensitive template is never penalized; a rapid second candidate jumps the score to $S_P = 2.0 > \theta$ and immediately triggers lockdown. This is a deliberate trade-off. A single unpadded probe cannot resolve a high-entropy secret (by Equation 16 below, its success is bounded by the prior guessing probability p_B), so Mirage trades an exploitable, unbounded timing channel for one bounded guess while preserving the benign first-access latency that isolation defenses destroy. A deployment that wants strictly zero first-visit leakage can instead set $G = 0$ ($\theta < \omega_{\mathsf{sens}}$), padding the first sensitive access; Equation 4 keeps either choice explicit and auditable. Section VI-E evaluates the low-and-slow adversary that the grace budget admits.

D. Emission Obfuscator

Naive first-token padding is unsafe for streaming APIs. Suppose the first token is held for Δ seconds while the backend continues decoding. Several later chunks may accumulate behind the sleep and then be released in a burst, producing abnormally small ITLs. Mirage therefore implements *stream time shifting*: it delays the client-visible emission schedule while leaving the GPU execution schedule untouched.

Let a_i be the gateway arrival time of the i-th streamed chunk from the inference engine, measured on a monotonic clock, and let t_0 be request arrival time. The physical TTFT is $t_{\mathsf{actual}} = a_1 - t_0$. In Phase 2, Mirage samples a target first-token latency τ from the calibrated miss distribution and computes

$$\Delta_{\mathsf{pad}} = \max(0, \tau - t_{\mathsf{actual}}). \tag{5}$$

The emitted time of every chunk is

$$e_i = a_i + \Delta_{\mathsf{pad}}, \qquad i = 1, 2, \ldots \tag{6}$$

so the observed first-token latency is $e_1 - t_0 = \max(t_{\mathsf{actual}}, \tau)$, while every inter-token gap is preserved:

$$e_i - e_{i-1} = a_i - a_{i-1}, \qquad i \ge 2. \tag{7}$$

Mirage realizes Equation 6 inside the tokenizer manager. When a request is admitted, Mirage records a monotonic request timestamp in the per-request state. Immediately before each streamed output chunk reaches the HTTP emission path, Mirage stamps the chunk with a GPU-side arrival timestamp. On the first chunk, the gateway computes t_{actual}, reads prompt-token and cache-hit metadata from the scheduler, consults the Privacy Odometer, records the observation for calibration, and stores Δ_{pad} in the request state. On every later chunk, the gateway releases the chunk at its stamped arrival time plus the same stored offset.

From per-request to per-batch equalization. The time shift above equalizes the emission surface at the level of a *single* request, making its first-token latency miss-like irrespective of the cache bit. A shared backend exposes one further observable: when an attacker co-submits several probes under the same prefix, a longest-prefix-match scheduler serves them in prefix-priority order, so the *order* in which their responses return reflects which candidate matched the victim's cache. Mirage closes this cross-request channel under the *same* phase gate that governs the time shift.

The gateway groups the concurrent responses of each locked-down prefix, identifying a group by the metered prefix hash, the same anchor P the odometer scores; all Phase-2 probes of one protected prefix therefore land in one group, from whatever account or connection they were submitted. Within a group, responses are held on an adaptive barrier that releases once the group has been quiet for a short settle interval, with an upper bound so that a stalled response cannot hold the group. At release, every response draws an independent uniform offset, and independent draws induce a uniformly random emission permutation: the order carries no information about which candidate hit. Two properties make this step cheap and safe. First, the barrier only ever *adds* delay on top of the per-request clamp: a grouped response is emitted at the later of its own

miss target τ and the group's release time, so the magnitude guarantee of Lemma 2, that every emitted first token sits at or above the miss tail, is untouched; the batch step removes the order signal without reopening the magnitude one. Second, the hold is confined to attack traffic: benign requests (Phase 0/1) are never grouped or reordered, and the responses that do wait belong to a prefix already under active fan-out and already being padded toward the same miss target. An attacker can escape the group only by spacing its probes out, which forfeits the co-submitted ordering the channel needs; each isolated probe then faces the per-request time shift alone. Appendix G confirms the mechanism still closes the order channel under real batched load (MRR=16), with the magnitude attacks unaffected.

Choosing the emission target. It remains to choose the target latency τ in Equation 6. Because the whole-stream shift already erases every inter-token cue (Lemma 2), only the emitted first-token law matters. Mirage's emitter is a one-sided clamp to a calibrated upper miss quantile Q,

$$e_1 - t_0 = \max(t_{\mathsf{actual}},\, Q), \qquad Q = Q_q\big(\widehat{F}_{\mathsf{miss}}\big), \tag{8}$$

where Q is the $(1+\eta)$-scaled q-th quantile ($q = 0.97$) of the online miss distribution $\widehat{F}_{\mathsf{miss}}$. Mirage applies this clamp to all requests under a Phase-2 prefix, regardless of their actual cache outcome. Consequently, physical hits and approximately 97% of genuine misses collapse onto the exact same latency floor Q. Because no unpadded baseline exists under a locked prefix, hit and miss candidates become statistically indistinguishable, rendering multi-probe statistical averaging futile (validated by the adaptive distinguisher in Section VI-E).

Formally, Lemma 2 bounds the attacker's residual advantage by the equalization gap δ_{eq}. Although clamping to the 97-th percentile incurs higher latency than padding to the mean miss latency, it is necessary: as shown in Section VI-F, lower-overhead distribution-matching emitters fail to reliably mask tail discrepancies and leave exploitable residuals. Clamping thus provides robust, worst-case indistinguishability while remaining substantially faster than cache isolation (Section VI-C).

E. Dynamic Miss Estimator

To ensure robust indistinguishability, the target padding latency τ must dynamically track the true cache-miss distribution under fluctuating server loads, batch sizes, and queueing delays. Static calibration is insufficient, as it risks under-padding during load spikes or adding excessive latency when the server is idle. Mirage resolves this by coupling an affine latency model with online regression.

Affine Latency Modeling. Cold-miss TTFT does not scale purely proportionally with prompt length L. Instead, it exhibits an affine relationship:

$$\mathrm{TTFT} \approx a + bL, \tag{9}$$

where the intercept a captures constant system overheads (e.g., tokenization, scheduling, and kernel launch latency), while the slope b reflects the marginal prefill cost per token. Because a strictly proportional model severely misestimates the execution time across varied prompt lengths, Mirage estimates latency directly in raw TTFT space.

Online Estimation and Target Generation. For each request that experiences a full cache miss (`cached_tokens` $= 0$), Mirage logs the observed (L, TTFT) tuple into a sliding window. It continuously updates the parameters (a, b) via streaming least-squares regression and computes the residuals $R = \mathrm{TTFT} - (a + bL)$. The upper quantile of these residuals, $Q_q(R)$, captures the real-time tail behavior of cache misses. For a protected request of length L, the target latency τ is computed as:

$$\tau = \max\Big\{(a+bL)(1+\eta),\ \big(a+bL+Q_q(R)\big)(1+\eta)\Big\} + \epsilon, \tag{10}$$

where η is a safety margin, q is the target quantile, and $\epsilon \sim \mathcal{U}(0,\sigma)$ injects bounded random jitter to eliminate deterministic timing signatures.

Cold-Start and Fallback Mechanisms. To guarantee protection when the regression window is still filling or contains degenerate prompt lengths, Mirage maintains secondary Exponential Moving Average (EMA) tracks for latency:

$$\mu_z(t) = \beta\mu_z(t-1) + (1-\beta)x_t, \quad z \in \{\mathsf{hit}, \mathsf{miss}\}. \tag{11}$$

Pure misses feed the miss track, while partial hits update the hit track after subtracting the estimated cost of their uncached suffix. During initial system warmup, Mirage adopts conservative default bounds, ensuring that requests remain fully protected from the moment the server boots.

V. Security and Performance Analysis

A. The Importance of the Fan-out Signal (ϕ)

Mirage's threshold is meaningful precisely because a timing attack must spend many candidate probes, and therefore open many novel fan-out branches under the same prefix, to resolve a secret. We formalize this fan-out requirement with a standard information-theoretic lower bound.

Theorem 1 (Fan-out lower bound)**.** *Let S^* be the unknown N-token secret continuation after prefix P. Let $\mathbf{O} = (O_1, \ldots, O_k)$ be the timing observations from k adaptive oracle queries. If each query has channel capacity at most C bits, then any attacker with recovery error at most ε must satisfy*

$$k \geq \frac{H(S^*|P) - h_2(\varepsilon) - \varepsilon \log_2(M-1)}{C}, \tag{12}$$

where $M = |\mathrm{supp}(S^*)|$ is the support size of S^*, i.e., the number of distinct candidates the secret continuation could take and the attacker must disambiguate, and h_2 is binary entropy. As $\varepsilon \to 0$, the lower bound tends to $H(S^*|P)/C$.

Proof. By the chain rule for mutual information, adaptivity does not let the attacker exceed the per-query channel capacity:

$$I(S^*; \mathbf{O}|P) \leq \sum_{i=1}^{k} I(S^*; O_i | P, O_{<i}) \leq kC. \tag{13}$$

Fano's inequality gives

$$H(S^*|\mathbf{O}, P) \leq h_2(\varepsilon) + \varepsilon \log_2(M-1). \tag{14}$$

Since $H(S^*|\mathbf{O}, P) = H(S^*|P) - I(S^*; \mathbf{O}|P)$, rearranging yields Equation 12. The limiting statement follows because both $h_2(\varepsilon)$ and $\varepsilon \log_2(M-1)$ go to zero for fixed finite M. □

How large is the per-query capacity C? The binary hit/miss verdict alone gives $C \leq 1$ bit, but the observable is a *continuous* TTFT whose magnitude also encodes how much of the prompt was cached. Since identifying the exact match depth among L possible token positions resolves $\log_2 L$ bits of uncertainty, an *unpadded* probe can carry up to $O(\log L)$ bits about the matched-prefix length. Mirage's two phases price this honestly. In the grace phase, probes are unpadded, so Theorem 1 must be read with this larger C, which is exactly why the grace budget is kept small (G=1). In lockdown, the clamp pushes every emitted first token to the miss tail, erasing the magnitude together with the hit/miss bit, so the per-probe capacity collapses to the equalization gap δ_{eq} bounded below (Lemma 2). Theorem 2 then composes the two phases: a few grace probes at full capacity, and every further probe at only δ_{eq}.

A complementary probability view turns the bound into a parameter rule. Let

$$p_B = \max_s \Pr[S^* = s|P] \tag{15}$$

denote the attacker's maximum prior probability of correctly guessing the exact protected secret (S^*) given only the known context prefix (P). In essence, p_B quantifies the attacker's best-case baseline success rate *before* observing any timing side-channel signals, leveraging their strongest language model or background knowledge to predict the most likely continuation s. Let $H_\infty(S^*|P) = -\log_2 p_B$ be the corresponding conditional min-entropy, which formally quantifies this fundamental unpredictability of the secret. Since Mirage allows an attacker to evaluate at most $K_{\max}(P)$ distinct Phase-1 candidates before triggering lockdown, a union bound yields the maximum probability of extracting the secret without being penalized:

$$\begin{aligned}\Pr[\mathsf{Phase\ 1\ success}] &\leq \min\{1, K_{\max}(P)p_B\} \\ &= \min\{1, K_{\max}(P)2^{-H_\infty(S^*|P)}\}.\end{aligned} \tag{16}$$

Therefore, a deployment with a strictly defined target Phase-1 leakage probability α should configure its grace budget as:

$$G = K_{\max}(P) \leq \left\lfloor \alpha\, 2^{H_\infty(S^*|P)} \right\rfloor \tag{17}$$

and subsequently set $G\,\omega_{\mathsf{sens}} \leq \theta < (G+1)\omega_{\mathsf{sens}}$. Our primary configuration ($\theta = 1.0, \omega_{\mathsf{sens}} = 1.0$) is the case $G = 1$: at most one unpadded candidate is admitted, so $\Pr[\mathsf{Phase\ 1\ success}] \leq p_B$, the prior guessing probability. After lockdown (Phase 2), the emitted timing distribution is forcefully equalized to the calibrated miss distribution. Consequently, the protected hit/miss bit contributes zero additional timing advantage in the ideal equalization model, reducing any residual success purely to the attacker's non-timing prior guessing (p_B).

B. The Security of the Time-Decay Factor (γ)

The time-decay factor γ is not intended to let an attacker probe forever at a low rate. KV cache entries are volatile data-plane state. Under normal multi-tenant load, cache eviction can be modeled as a stochastic process with rate λ. If evictions of the target prefix are approximated by a Poisson process, the probability that the target cache remains alive for duration T is

$$\Pr[\mathsf{survive}\ T] = e^{-\lambda T}. \tag{18}$$

An attacker who needs k candidate probes and waits average interval Δt spends approximately $T = k\Delta t$ time, giving

$$\Pr[\mathsf{cache\ survives\ attack}] \approx e^{-\lambda k \Delta t}. \tag{19}$$

Large Δt helps the odometer decay, but it exponentially reduces the chance that the victim's cache state survives long enough to be queried. Small Δt preserves cache survival, but then $\gamma^{\Delta t} \approx 1$ and the score in Equation 2 quickly crosses θ. This creates the adversary's spatio-temporal dilemma: to keep the cache alive, the attacker must probe quickly; probing quickly exposes the high-fan-out pattern that Mirage meters.

This also gives a direct rule for choosing γ. If a prefix has current score $S_0 > \theta$, the idle time needed to decay back below the grace boundary is

$$T_{\mathsf{decay}}(S_0, \theta, \gamma) = \frac{\ln(\theta/S_0)}{\ln \gamma}. \tag{20}$$

For a prefix that has just entered lockdown at $S_0 = 2$ (two rapid novel probes), our default $\theta = 1.0$ and $\gamma = 0.995$/s require about 138 s of idle time before it decays back below the grace boundary. Thus γ should be selected so that T_{decay} is longer than the time window over which a provider expects a sensitive KV entry to remain reliably resident under normal load, or longer than the cadence at which repeated probes can be rate-limited. Decreasing γ shortens memory and improves benign liveness; increasing γ lengthens lockdown and closes low-and-slow repetition more aggressively.

C. Conditional Equalization Guarantee

Theorem 1 shows that resolving a secret needs many novel probes; Mirage admits those probes only once the prefix is in lockdown, where their timing is equalized. We state that guarantee conditionally and then compose it with Theorem 1.

Fix a metered prefix P and let $B \in \{\mathsf{hit}, \mathsf{miss}\}$ indicate whether a probe's continuation matches the victim's cached prefix, under an *arbitrary* prior $\pi = \Pr[B = \mathsf{hit}]$. The attacker observes the whole emitted stream

$$\mathcal{T} = (o_1, g_2, \ldots, g_n), \quad o_1 = e_1 - t_0, \quad g_i = e_i - e_{i-1},$$

i.e. the first-token latency and every inter-token gap. Write $\mathcal{L}_b^o$ and $\mathcal{L}_b^{\mathcal{T}}$ for the laws of o_1 and of $\mathcal{T}$ given $B = b$.

Assumption 1 (Equalization regime)**.** (A1) Gating and lockdown: $\omega_{\mathsf{sens}}(P) > 0$*, every distinct candidate probe maps to the same metered prefix* P*, and* P *is in Phase 2, i.e.* $S_P(t) > \theta$*.* (A2) Decode-schedule conditional independence: *there is a Markov kernel* K *with* $\mathcal{L}(g_{2:n} \mid B = b, o_1 = o) = K(\cdot \mid o)$ *for a.e.* o *and both* $b \in \{\mathsf{hit}, \mathsf{miss}\}$. (A3) First-token equalization: $\mathrm{TV}(\mathcal{L}^o_{\mathsf{hit}}, \mathcal{L}^o_{\mathsf{miss}}) \leq \delta_{\mathsf{eq}}$.

(A2) is where the whole-stream claim is earned. Mirage applies a *single* offset Δ_{pad} per chunk, so by (6) every gap satisfies $e_i - e_{i-1} = a_i - a_{i-1}$: the emitter never manufactures a burst. Gaps are not *unconditionally* independent of B (decode cadence and prefill are load-driven); (A2) asks only that *given* o_1, B adds nothing further. The dynamic miss estimator (Section IV-E) buys exactly this: τ is resampled from the *current* miss window, so a padded hit and a natural miss

sharing o_1 share the load regime that produced their gaps. It is falsifiable: the distinguisher of Section VI-E tests it directly.

Lemma 1 (Trace reduction)**.** *Under (A2),* $\mathrm{TV}(\mathcal{L}^{\mathcal{T}}_{\mathsf{hit}}, \mathcal{L}^{\mathcal{T}}_{\mathsf{miss}}) = \mathrm{TV}(\mathcal{L}^{o}_{\mathsf{hit}}, \mathcal{L}^{o}_{\mathsf{miss}})$.

Proof. See Appendix A. □

Lemma 2 (Bounded advantage and capacity)**.** *Under Assumption 1, for a single probe:* (a) *every (possibly randomized) test on the whole trace* $\mathcal{T}$ *has advantage at most* δ_{eq} *in deciding* B*;* (b) *for* every *prior* π*,* $I(B;\mathcal{T}) \le h_2(\pi)\,\delta_{\mathsf{eq}} \le \delta_{\mathsf{eq}}$ *bits, so the per-query capacity satisfies* $C_{\mathsf{eq}} := \max_\pi I(B;\mathcal{T}) \le \delta_{\mathsf{eq}}$*. As* $\delta_{\mathsf{eq}} \to 0$ *both vanish.*

Proof. See Appendix A. □

Part (b) says: with probability $1-\delta_{\mathsf{eq}}$ the emitted latency is drawn from a law common to hits and misses, hence uninformative; only with probability δ_{eq} can it betray the bit. It subsumes the uniform-prior statement $I(B;o_1) = \mathrm{JSD} \le \delta_{\mathsf{eq}}$ ($\pi = \frac{1}{2}$, $h_2 = 1$) and, unlike it, holds for *every* prior, which is exactly what Theorem 2 needs.

Corollary 1 (Repeated measurement)**.** *If the attacker repeats a probe* R *times with traces conditionally i.i.d. given* B*, its advantage is at most* $1-(1-\delta_{\mathsf{eq}})^R$ *and* $I(B;\mathcal{T}^{(R)}) \le h_2(\pi)\big[1-(1-\delta_{\mathsf{eq}})^R\big]$.

Proof. See Appendix A. □

Theorem 2 (Odometer–equalization composition)**.** *Let an attacker issue* k *probes under* P*, of which* N_1 *are admitted in Phase 1, with Assumption 1 holding for every Phase-2 probe. Then recovering* S^* *with error at most* ε *requires*

$$k \;\ge\; N_1 \;+\; \frac{H(S^*\,|\,P) - h_2(\varepsilon) - \varepsilon\log_2(M-1) - N_1}{\delta_{\mathsf{eq}}}. \tag{21}$$

Proof. See Appendix A. □

Equation (21) *refines* rather than contradicts (12): setting $N_1 = 0$ and $\delta_{\mathsf{eq}} = C$ recovers Theorem 1 exactly, and since $\delta_{\mathsf{eq}} \le 1$, the composed bound is never weaker. The fundamental difference lies in the information accounting. While Theorem 1 assumes every probe leaks the same maximum capacity, Theorem 2 models Mirage's two-phase reality: it charges the initial N_1 grace probes at full capacity, but restricts every subsequent probe to a minimal leakage of δ_{eq}.

Crucially, this mathematical composition mirrors how Mirage's three physical mechanisms collaborate to corner the attacker: First, the *odometer* strictly caps N_1, allowing at most $K_{\max}(P)$ unpadded probes per decay window (Equation 4). Second, *equalization* drastically inflates the attack cost during lockdown, forcing the adversary to spend $1/\delta_{\mathsf{eq}}$ probes to extract just one additional bit. Finally, natural *cache residency* (Equation 19) closes the low-and-slow evasion loophole, physically bounding how many decay windows an attacker can patiently wait out before the targeted cache entry is evicted.

The guarantee is conditional: it degrades if the gate misfires or probes split across metered prefixes (A1), if the schedule leaks beyond o_1 (A2), or if $\delta_{\mathsf{eq}} > 0$ (A3); Appendix B in turn bounds this gap δ_{eq} by the tunable clamp knobs (q,η), pricing security directly in padding latency. Finally, δ_{eq} is a *measured* quantity, not a bound over all conceivable tests: any concrete distinguisher only lower-bounds the true total variation. Because the optimal test on a balanced population attains accuracy $\frac{1}{2}(1+\mathrm{TV})$, a distinguisher's held-out *balanced accuracy* acc (the mean of its true-positive and true-negative rates) yields the estimate $\widehat{\delta}_{\mathsf{eq}} = 2\,\mathrm{acc} - 1$, which grows as the distinguisher strengthens; a residual that survived a stronger future test would raise $\widehat{\delta}_{\mathsf{eq}}$ and, through (21), the number of probes that it implies. Finally, Appendix B shows that benign traffic remains unpadded and GPU demand is zero. Proposition 1 bounds $\delta_{\mathsf{eq}} \le (1-q) + \varepsilon_{\mathsf{est}} + p_{\mathsf{slow}}$, directly linking the deployment knobs (q,η) to the empirically measured residual $\widehat{\delta}_{\mathsf{eq}}$.

VI. Experiments

We integrate Mirage into SGLang [10], a state-of-the-art serving framework featuring a native radix-tree prefix cache. Our evaluation answers three questions: (Q1) how effectively does Mirage suppress timing and response-order side channels relative to these baselines; (Q2) what serving-performance cost does it impose on benign public-prefix traffic; and (Q3) does it remain robust against an adaptive adversary with full knowledge of the mechanism (Section VI-E)?

A. Experimental Setup

Our full experimental setup, the hardware and serving configuration, the models and datasets, and Mirage's defense configuration, is deferred to Appendix C; we summarize the attacks, baselines, and metrics here.

Attacks and baselines. We evaluate three KV-cache timing attacks that span the leakage spectrum. *EarlyBird* [16] recovers a single next token from per-request TTFT magnitude. *InputSnatch* [17] recovers a structured (name, condition) field from the same magnitude signal; a matching field differs by only a few prefix tokens. *OptiLeak* [26] exploits the response *order* of a co-submitted probe batch under SGLang's longest-prefix-match scheduling, a strong order channel. We compare against four points: an undefended *No Defense*, *Cache Partition* [20] (per-tenant radix-tree namespaces), *CacheSalt* [21] (per-request salts/extra cache keys), and *SafeKV* [22] (a sensitivity-aware proxy that isolates prompts classified as sensitive).

Metrics. We quantify attack efficacy using $\mathrm{ASR@}N$, defined as the attack success rate within a budget of N candidate queries. An evaluation unit represents one target to recover: a hidden private entity for EarlyBird and InputSnatch, and a sensitive token position for OptiLeak. Specifically, EarlyBird searches over the full candidate pool of 467 first-tokens, while InputSnatch performs end-to-end recovery over the complete 500×500 (name, condition) dictionary without artificial ground-truth injection. For OptiLeak, candidates are evaluated in co-submitted batches of ≤ 50 along the model's prediction ranking up to rank 10,000. We report 95% percentile bootstrap confidence intervals for every headline number; the resampling methodology and per-cell intervals are in Appendix D.

B. Security Effectiveness

Table I reports the ASR@N across the three evaluated models. Without defenses, all three timing channels leak significantly. This vulnerability worsens with model scale, as larger models exhibit longer prefill times that create a sharper latency gap between cache hits and misses. For example, OptiLeak's order channel achieves an ASR@10k of 67.6–81.8%. EarlyBird's magnitude channel reaches 92.4% on DeepSeek-14B, dropping to 6.2% on the smaller Qwen1.5-1.8B. InputSnatch, which recovers a complex multi-token field, follows the exact same trend. When evaluated over its full dictionary to avoid artificially masking the attack, its end-to-end recovery climbs from the 0.0% guessing floor on Qwen (where the minimal TTFT gap is lost in system noise) all the way to 59.4% on the DeepSeek-14B model.

Mirage mitigates all three attacks using a single, unmodified policy. Across all models, OptiLeak's success rate drops to exactly 0.0%, while EarlyBird falls to near-zero (0.0–0.2%). InputSnatch is similarly suppressed to $\leq 0.4\%$ end-to-end. Although a negligible residual timing signal survives for individual tokens on larger models due to the equalization gap, the combined fields remain strictly at chance, ensuring that full end-to-end extraction consistently fails.

Baseline defenses like Cache Partition and CacheSalt also successfully suppress these channels to the random guessing floor (0.0–1.0% across all attacks and budgets). However, they achieve this security exclusively by disabling cross-tenant cache reuse entirely, forcing redundant model recomputation. SafeKV, which instead shares selectively, leaves a slightly larger residual (up to 2.0% EarlyBird on Qwen1.5-1.8B) wherever its classifier misjudges a sensitive prefix as public. Ultimately, Mirage achieves the same near-zero ASR as these isolation defenses, but crucially does so while preserving the computational benefits of cache sharing for benign traffic.

C. Performance Overhead

We evaluate serving overhead using public Alpaca prompts generated by 500 virtual users under an open-loop Poisson arrival process. We sweep the offered load from 1 to 8 requests per second, recording the median TTFT and goodput at each point (Figure 2). To cleanly isolate the defense's inherent overhead from severe system queueing delays, all single-value latency metrics reported below are measured at each model's peak unsaturated load. This is defined as the maximum rate where goodput maintains at least 90% of the offered load (1, 2, and 6 req/s for the DeepSeek-14B, Llama-3-8B, and Qwen1.5-1.8B models, respectively).

For public traffic, Mirage imposes negligible overhead. Its median TTFT remains within 1.5% of the undefended baseline (55.2 ms vs. 54.4 ms in a controlled 300-request trial). In contrast, strict isolation mechanisms like Cache Partition and CacheSalt penalize all traffic indiscriminately, inflating DeepSeek-14B's median TTFT by 44–46% and causing multi-fold latency increases on Qwen1.5-1.8B at higher loads. SafeKV, a selective isolation system, matches Mirage and the undefended baseline here, provided its detection runs off the critical path and correctly leaves public traffic untouched. Ultimately, on purely public workloads, Mirage matches the efficiency of ideally configured selective sharing while thoroughly outperforming unconditional isolation. This advantage widens sharply under contended sensitive-shared workloads: when the GPU is saturated, isolation's redundant recomputation inflates TTFT by roughly 3× while Mirage reuses the cached prefix and tracks No Defense (Appendix F).

Having established Mirage's efficiency on public workloads, we now evaluate the precision of its behavioral trigger on sensitive traffic. To demonstrate that this padding is applied exclusively when necessary, we isolate the trigger mechanism by comparing Mirage at its default grace budget (G=1) against two absolute bounds: No Defense (always off) and a Static Pad configuration (G=0, where every sensitive request is unconditionally padded).

We measure the median TTFT of a sensitive cache access under two conditions: a single legitimate query (benign first-visit) and the exact same query issued during an active fan-out attack. By keeping the baseline request rate low, we allow the odometer's time-decay to naturally spare genuine traffic while rapidly locking down deliberate probing bursts. As shown in Table II, No Defense processes both conditions at the native ≈150 ms speed but leaves the timing channel wide open (ASR up to 92%, Table I). Conversely, the Static Pad configuration securely closes the channel but severely penalizes the user experience, inflicting a 417 ms delay even on the benign first-visit. Mirage effortlessly bridges this gap. It serves the benign user at the pristine 150 ms latency (recording zero Phase-2 events) while dynamically applying the 418 ms obfuscation solely when the prefix is under attack.

Throughput headroom. The latency sweep above uses one in-flight request (MRR=1), the adversary-favorable setting that sharpens the timing signal. We now set MRR=16 to run the GPU as a real batched server and stress throughput. Mirage spends no GPU cycles on the defense (the pad is at the gateway and the shared prefix is reused), whereas isolation recomputes the shared prefix for every tenant, burning cycles that could serve new requests. On Llama-3-8B (Figure 3), Mirage's goodput tracks No Defense along the ideal line while per-tenant isolation saturates near 14 req/s; at 24 req/s Mirage sustains about 1.6× its throughput, matching the capacity ratio that Theorem 4 predicts. Because the pad never stalls the GPU, padding the sensitive share of this traffic (5% here) leaves Mirage's goodput indistinguishable from No Defense. This headroom, distinct from the latency result of Figure 2, is what isolation forfeits and Mirage retains.

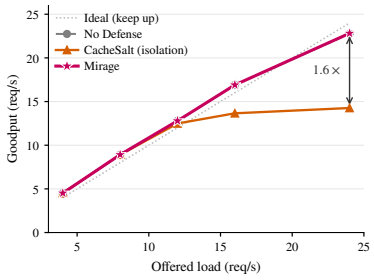

Fig. 3. Goodput vs. offered load (Llama-3-8B). Mirage tracks No Defense along the ideal line; per-tenant isolation saturates ∼1.6× earlier.

Memory and control-plane cost. The Privacy Odometer lives

TABLE I
ATTACK SUCCESS RATE (ASR@N, %) ACROSS MODEL SCALES AND DEFENSES.

Model	Defense	EarlyBird ↓			InputSnatch ↓	OptiLeak ↓		
		ASR@100	ASR@1k	ASR@10k	ASR@100–10k	ASR@100	ASR@1k	ASR@10k
DeepSeek-14B	No Defense	21.6	92.4	92.4	59.4	25.5	41.2	68.8
	Cache Partition	0.0	0.5	0.5	0.0	0.0	0.0	0.0
	CacheSalt	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	SafeKV	0.0	0.5	0.5	0.0	0.0	0.0	0.0
	Mirage	**0.0**	**0.2**	**0.2**	**0.4**	**0.0**	**0.0**	**0.0**
Llama-3-8B	No Defense	8.2	33.4	33.4	34.2	29.2	48.8	81.8
	Cache Partition	0.0	1.0	1.0	0.0	0.0	0.0	0.0
	CacheSalt	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	SafeKV	0.0	0.5	0.5	0.0	0.0	0.0	0.0
	Mirage	**0.0**	**0.0**	**0.0**	**0.4**	**0.0**	**0.0**	**0.0**
Qwen1.5-1.8B	No Defense	2.2	6.2	6.2	0.0	24.1	38.9	67.6
	Cache Partition	0.0	0.5	0.5	0.0	0.0	0.0	0.0
	CacheSalt	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	SafeKV	0.0	2.0	2.0	0.0	0.0	0.0	0.0
	Mirage	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**

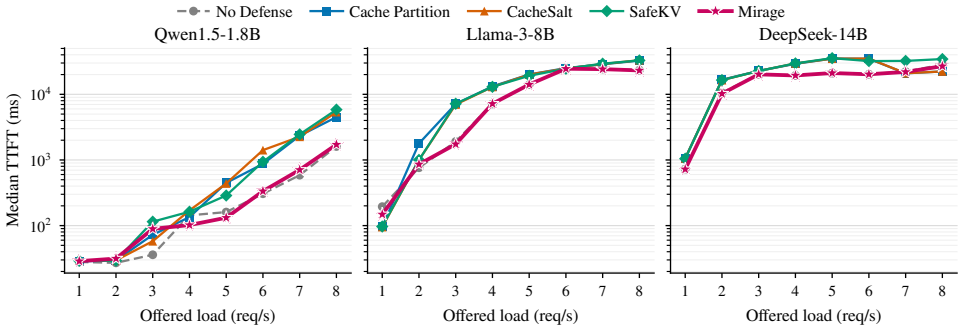

Fig. 2. Median TTFT (log scale) vs. offered load on public traffic. Mirage (pink) closely tracks No Defense (grey dashed, largely occluded); isolation baselines sit above. Curves converge only in the saturated tail, where queueing dominates.

TABLE II
MEDIAN TTFT (MS) OF A SENSITIVE ACCESS (LLAMA-3-8B): MIRAGE PADS ONLY UNDER ATTACK; PAD-ALL PADS EVERY SENSITIVE REQUEST.

Configuration	Benign first-visit TTFT (ms)	Under attack TTFT (ms)	Attack ASR
No Defense	149	151	leaks
Mirage (G=1)	**150**	418	≈ 0
Naive pad-all (G=0)	417	416	≈ 0

entirely in the host tokenizer manager and adds *zero* GPU memory. Each tracked prefix costs about 520 B plus ≈ 100 B per distinct branch, so tracking 10,000 active sensitive prefixes uses ≈ 35 MiB, negligible beside the weights and KV cache. On the critical path, a full odometer decision (sensitivity scan, novelty hash, decayed-score update, phase gate) costs a median ≈ 6.4 μs per request, roughly four orders of magnitude below a typical ≈ 150 ms TTFT; this is the constant control-plane term c_{ctrl} that Theorem 3 proves is Mirage's only latency cost on benign traffic.

D. Generalization to Semantic Caches

We now test whether the defense generalizes beyond KV/prefix caches to a *semantic cache*. A GPTCache-style semantic cache embeds each query and returns the stored response of a sufficiently similar earlier query, so a hit answers in milliseconds while a miss runs the full model. The hit/miss gap is therefore a whole response (70–450 ms in our runs), much larger than the per-token prefill gap of Section VI, which makes the undefended channel wide open and the layer a harder test for the defense.

We lift the same two-phase design to this layer. Only the novelty signal has to change, because an embedding space has no exact continuation hash to key on. Consider first how the attacker behaves. To pull a secret out of a semantic cache, an attacker has to work a small neighborhood of the embedding space hard: it fires a rapid burst of queries that are all worded almost the same, sweeping the candidate answers one after another. A benign user does the opposite: they ask about the one entity they care about, rephrasing it at most a few times. This gives two behavioral tells that a benign user does not produce together, a sudden *density* of near-duplicate queries in one region, and a sweep *across distinct entities*. The separation is wide in practice: in our corpus, benign paraphrases of one topic have an average pairwise cosine of 0.835, while an attacker's probes of different entities sit at 0.445.

Mirage-Semantic meters exactly these two tells with two matching components, *semantic-differential hashing* for the entity sweep and *local density tracking* for the burst. The detector maintains a set of *entity representatives*. If a new query's similarity to an existing representative exceeds a threshold (s_e=0.80), it collapses onto that entity; otherwise, it registers as a novel entity. We then track the time-decayed density of distinct entities queried within a region:

$$D_{\mathsf{ent}}(t_j) = \gamma^{\,t_j - t_{j-1}} D_{\mathsf{ent}}(t_{j-1}) + \phi_{\mathsf{ent}}(q_j), \qquad (22)$$

where $\phi_{\mathsf{ent}}(q_j) = 1$ only when a query opens a new entity. Benign paraphrases map to the same representative and never raise the risk score. However, an attacker's rapid sweep across

TABLE III
SEMANTIC-CACHE GENERALIZATION. ASR BASELINES $50/10/2.5\%$; AUC 0.5 = CLOSED; Δt = MEAN MISS−HIT GAP (MS), ≈ 0 ONCE EQUALIZED (SMALL NEGATIVES ARE NOISE).

		No Defense			Mirage-Semantic		
Attack Objective	**Backend**	ASR↓	AUC→.5	Δt ↓	ASR↓	AUC→.5	Δt ↓
Membership	Qwen1.5-1.8B	99.2	1.00	71.6	54.1	0.55	-0.6
	Llama-3-8B	99.2	1.00	246.8	46.8	0.48	-0.9
	DeepSeek-14B	99.2	1.00	448.8	53.7	0.47	4.3
Closed-world	Qwen1.5-1.8B	96.0	1.00	70.0	11.0	0.51	0.0
	Llama-3-8B	99.0	1.00	243.2	13.0	0.54	0.2
	DeepSeek-14B	98.0	1.00	444.3	4.0	0.49	-0.3
Open-world	Qwen1.5-1.8B	62.0	0.99	68.8	3.0	0.48	-0.2
	Llama-3-8B	58.0	0.99	240.3	2.0	0.50	-0.1
	DeepSeek-14B	54.0	0.99	437.6	1.0	0.47	-0.3

distinct entities quickly pushes D_{ent} past a safety threshold (γ=0.98/s, floor 4). Once triggered, the region enters lockdown, and hit latencies are padded up to the miss distribution.

We evaluate this mechanism on a 40-topic medical corpus using an `all-MiniLM-L6-v2` embedder and the three backend models from Table I. Because standard KV-cache attacks do not apply to semantic caches, we port their underlying objectives: (i) *membership* (detecting whether a target topic was queried), (ii) *closed-world* (recovering the exact topic among K=10 candidates), and (iii) *open-world* (recovering the topic among all K=40 candidates). We run 100 trials per objective. Since these are classification tasks, we evaluate success against random guessing baselines of 50%, 10%, and 2.5%, respectively, not the budget-indexed ASR@N of Table I, so Table III should not be compared cell-by-cell with Table I.

Table III shows the outcome. Undefended, the cache is a near-perfect oracle on every objective and backend (AUC ≥ 0.99; closed-world ASR 96–99% against a 10% baseline). Mirage-Semantic returns all three attacks to their random baselines (AUC ≈ 0.5, timing gap ≤ 5 ms) and pads *zero* benign traffic in our paraphrase and diverse-topic workloads, because a scan must spread across many entities while benign traffic does not. The defense principle therefore carries across cache layers.

E. Robustness to Adaptive Adversaries

We assume a strong white-box adversary who knows Mirage's full design and parameters (the scoring rule, two-phase policy, equalization target, and decay constant γ) and adapts accordingly, but is not granted access to Mirage's internal state: it cannot see whether a response was a genuine miss or a padded hit, nor read a prefix's current score or phase. It attacks the residuals the mechanism leaves behind in two ways: passively mining distribution shapes under lockdown, and actively manipulating the temporal decay.

Multivariate Distribution Distinguisher. Rather than compare mean latencies, the attacker takes $R \in \{10, 20\}$ measurements per candidate, extracts an 8-feature vector (mean, median, min, q10, q25, q75, q90, std), and trains an L2-regularized logistic regression to separate hits from misses, evaluated over 200 controlled hit/miss pairs with 5-fold cross-validation. We report the effective AUC $\max(\text{AUC}, 1-\text{AUC})$. Against an undefended server the channel is wide open and sharpens with repetition exactly as Corollary 1 predicts: CV AUC rises from 0.807 (R=10) to 0.962 (R=20, Table XII), while against Mirage the corollary's growth term $1-(1-\delta_{\mathsf{eq}})^R$ stays negligible at the measured $\widehat{\delta}_{\mathsf{eq}}$.

An 8-feature logistic model (R=20 repeats: mean, median, and tail quantiles of TTFT) is trained to separate padded hits from natural misses. Against Mirage's clamp emitter its effective AUC is 0.53 ± 0.01 over six runs. We report 0.53 rather than "chance" because the small gap is real: by the estimator of Section V it is a measured equalization gap $\widehat{\delta}_{\mathsf{eq}} = 2(0.53) - 1 \approx 0.06$, a residual of at most 0.06 bits per probe. This does not compose into a usable channel: by Theorem 2 recovering one high-entropy token position needs on the order of $1/0.06 \approx 17$ padded probes per bit, each still bounded by cache residency (Equation 19). The honest summary is near chance with a small, measured residual.

Whole-trace distinguisher. The TTFT distinguisher above tested only first-token features, so it left Assumption 1(A2) untested: that the post-first-token schedule, i.e. the *inter-token latencies* (ITLs), leaks nothing about the cache bit beyond o_1. We therefore build a stronger distinguisher that observes the *entire* streaming trace, the first-token latency *and* every inter-token gap, by requesting 16 tokens and timestamping each. Crucially, to isolate a pure *timing* channel from a content artifact, the attacker always sends the *same* probe X; we vary only the cache state, priming X for a hit and priming a sibling $P\|\text{wrong}$ (a realistic partial miss: the shared prefix P stays warm, only the probed candidate is cold) for a miss. The generated text is therefore identical and any residual is pure timing. We extract TTFT features, ITL features (per-position gaps, dispersion, and lag-1 autocorrelation), and the raw per-trace vector, and train gradient-boosted and random-forest classifiers in addition to logistic regression, reporting 5-fold effective AUC. This is a strictly stronger family than the 8-feature TTFT logistic model, and it tests A2 head-on.

Table IV evaluates this distinguisher on Llama-3-8B (excluding Qwen1.5-1.8B due to its inherently weak baseline channel; its undefended EarlyBird is only 6%). The results yield two critical insights. First, the ITL channel is highly exploitable: without defenses, the ITL-only and whole-trace classifiers achieve AUCs of 0.90 and 0.99, respectively. This proves that a naive defense that only pads the first token while exposing the original decode cadence remains deeply vulnerable. Second, Mirage effectively neutralizes this threat. Its whole-stream time-shift holds the ITL-only (0.53), whole-trace (0.56), and raw per-trace (0.58) classifiers near random chance. Because Mirage resamples padded hits from the current miss load regime, the preserved ITLs carry no incremental signal beyond the TTFT-only baseline (0.58), empirically validating Assumption A2. We observe identical success on DeepSeek-14B, the model with the widest undefended separation (whole-trace AUC 0.98). Mirage collapses its AUCs down to 0.60–0.63.

Low-and-slow evasion. A patient adversary can instead wait out the odometer's time-decay and spread *distinct* candidates so each is served in the unpadded grace phase. We analyze this spread-candidate strategy and the exact-repeat variant, and show that both collapse to the guessing baseline in deployment,

TABLE IV
CONTENT-CONTROLLED WHOLE-TRACE DISTINGUISHER (EFFECTIVE AUC, 0.5 = CLOSED; FIXED PROBE, PURE TIMING). MIRAGE HOLDS TTFT, ITL, AND WHOLE-TRACE FEATURES AT CHANCE (A2).

Model	Defense	TTFT	ITL-only	Whole trace	Per-trace
Llama-3-8B	No Defense	0.98	0.90	0.99	0.99
	Mirage	**0.58**	**0.53**	**0.56**	**0.58**
DeepSeek-14B	No Defense	0.97	0.81	0.98	0.96
	Mirage	0.60	0.63	0.60	0.56

TABLE V
EMITTER ABLATION (EFFECTIVE AUC, n=3 RUNS). ONLY THE CLAMP CLOSES THE CHANNEL IN EVERY RUN; DISTRIBUTIONAL EMITTERS CUT OVERHEAD BUT LEAVE A RESIDUAL.

Emitter	Eff. AUC $\to$.5	Mean pad (ms) $\downarrow$
Clamp (Mirage default)	**0.53 $\pm$.01**	35
Full CDF (q_0=0)	0.67 $\pm$.04	10
Tail-conditioned (q_0=0.7)	0.55 $\pm$.06	12
Parametric log-normal (q_0=0.7)	0.58 $\pm$.10	23

in Appendix H.

F. Ablation Study

This section justifies our primary structural design choice: the shape of the emission target. Extensive parameter sweeps for all scalar hyperparameters (the grace budget G, repeat weight ρ, clamp quantile q, and margin η), alongside evaluations of low-and-slow adaptive evasion, confirm the robustness of our default configurations and are detailed in Appendix H.

Emission target. The clamp of Equation 8 over-pads every hit to the miss tail. A tempting alternative is to *match* the miss distribution so a fast hit is padded only to a typical miss, $\tau = F^{-1}_{\text{miss}}(U)$. We implemented and tested this family carefully, at n=3 runs of the multivariate distinguisher each (Table V). Matching the full law ($U \sim \text{Uniform}(0,1)$) is the cheapest emitter (10 ms mean pad) but leaks (effective AUC 0.67): in the adversarial setting genuine misses are cold full-prefill executions and are slower than a typical recent miss, so a full-law draw emits some hits *faster* than the cold misses and the distinguisher reads a "hit is faster" residual. Restricting the draw to the upper tail (a quantile floor q_0=0.7) or replacing the empirical window with a fitted parametric log-normal tail both *reduce* the residual but do not robustly remove it: they average AUC 0.55–0.58 with a large run-to-run standard deviation ($\pm$0.06–0.10) and leak on some runs, because reconstructing the cold-miss upper tail from a finite online window is inherently unreliable. Only the clamp, which sidesteps estimation by over-padding to the tail by construction, holds at chance (0.53 ± 0.01) in *every* run. Mirage therefore pays the clamp's extra overhead in exchange for robust indistinguishability; a low-overhead distributional emitter with the same guarantee remains open.

VII. RELATED WORK

LLM Timing Side-Channel Attacks. Recent work shows that AI serving optimizations can become timing oracles. EarlyBird [16] studies timing leakage in LLM serving and demonstrates prompt recovery from cache-dependent latency differences. InputSnatch [17] turns prefill-time differences into practical black-box input recovery attacks across model and hardware configurations. PromptPeek [18] shows that KV-cache sharing in multi-tenant serving can leak prompts by probing candidate continuations and observing cache-related behavior. Complementing these attacks, auditing work [19] measures prompt caching behavior in commercial APIs and shows that externally visible timing can reveal when cached prefixes are reused. Other LLM side channels exploit different observable signals, including token lengths in encrypted streams [27] and timing variations introduced by efficient inference mechanisms such as speculative decoding [28]. Mirage targets the KV-prefix-cache timing channel specifically, and assumes the attacker observes the full streaming trace rather than TTFT alone.

LLM Timing Side-Channel Defenses. The most direct defense is to prevent cross-tenant cache reuse. Cache partitioning [20] assigns each user an isolated prefix-cache namespace, eliminating cross-user timing interactions at the cost of losing shared-prefix benefits. Production systems can implement a related policy using cache salts or extra hash keys [21] so that only requests with the same trust boundary share cached blocks. These mechanisms are simple and strong, but they penalize benign workloads with common public templates or shared system prompts, the very traffic that caching is built to reuse.

Selective-sharing systems try to recover some performance by distinguishing sensitive from shareable cache entries. SafeKV [22], for example, combines privacy detection, unified public/private metadata, and runtime safeguards to restrict reuse of sensitive KV blocks while preserving safe sharing. Such designs narrow the blast radius of isolation, but cannot escape it: every prefix marked as sensitive by the classifier is still denied cross-tenant reuse, so benign users of that prefix pay the full recomputation cost. This makes the content classifier a single point of failure for both security (a missed prefix leaks completely) and performance (an over-broad label isolates benign traffic), and it leaves the attacker's defining behavior, the burst of distinct candidate probes under one prefix, entirely unexamined. Mirage instead keeps sensitive prefixes in the shared cache and reacts to that probing behavior, obfuscating only the emitted timing once a prefix's distinctive fan-out signature shows up.

VIII. CONCLUSION

In this paper, we introduced Mirage, mitigating KV Cache side-channels by shifting timing delays directly at the API gateway rather than pausing computation or enforcing strict spatial barriers. Supported by a conditional information-theoretic analysis and extensive evaluation, with designs such as correct sensitivity gating, consistent prefix metering, and well-calibrated equalization, Mirage attains security comparable to absolute cache separation against the probing attacks we evaluate, up to a small, measured residual and under stated assumptions, while adding near-zero latency on public traffic under high multi-tenant loads and paying only a bounded, quantified cost on sensitive traffic that is under active probing.

REFERENCES

[1] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, "Large language models in medicine," *Nature medicine*, vol. 29, no. 8, pp. 1930–1940, 2023.

[2] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "Bloomberggpt: A large language model for finance," *arXiv preprint arXiv:2303.17564*, 2023.

[3] J. Spataro, "Introducing microsoft 365 copilot," https://blogs.microsoft.com/blog/2023/03/16/introducing-microsoft-365-copilot-your-copilot-for-work/, 2023.

[4] Z. Yao, R. Yazdani Aminabadi, M. Zhang, X. Wu, C. Li, and Y. He, "Zeroquant: Efficient and affordable post-training quantization for large-scale transformers," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 168–27 183, 2022.

[5] X. Wei, Y. Zhang, X. Zhang, R. Gong, S. Zhang, Q. Zhang, F. Yu, and X. Liu, "Outlier suppression: Pushing the limit of low-bit transformer language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 17 402–17 414, 2022.

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.

[7] W. Wang, W. Chen, Y. Luo, Y. Long, Z. Lin, L. Zhang, B. Lin, D. Cai, and X. He, "Model compression and efficient inference for large language models: A survey," *arXiv preprint arXiv:2402.09748*, 2024.

[8] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.

[9] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*. New York, NY, USA: Association for Computing Machinery, 2023, pp. 611–626.

[10] L. Zheng, L. Yin, Z. Xie, C. L. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "SGLang: Efficient execution of structured language model programs," *Advances in Neural Information Processing Systems*, vol. 37, pp. 62 557–62 583, 2024.

[11] J. Juravsky, B. Brown, R. Ehrlich, D. Y. Fu, C. Re, and A. Mirhoseini, "Hydragen: High-throughput LLM inference with shared prefixes," 2024. [Online]. Available: https://arxiv.org/abs/2402.05099

[12] L. Ye, Z. Tao, Y. Huang, and Y. Li, "ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 11 608–11 620.

[13] OpenAI, "Prompt caching: Reduce latency and cost with prompt caching," 2024, accessed: 2026-05-24. [Online]. Available: https://platform.openai.com/docs/guides/prompt-caching

[14] Anthropic, "Prompt caching with claude," 2024, accessed: 2026-05-24. [Online]. Available: https://www.anthropic.com/news/prompt-caching

[15] DeepSeek, "Deepseek api docs: Deepseek api introduces context caching on disk," 2024, accessed: 2026-05-24. [Online]. Available: https://api-docs.deepseek.com/news/news0802/

[16] L. Song, Z. Pang, W. Wang, Z. Wang, X. Wang, H. Chen, W. Song, Y. Jin, D. Meng, and R. Hou, "The early bird catches the leak: Unveiling timing side channels in LLM serving systems," 2025. [Online]. Available: https://arxiv.org/abs/2409.20002

[17] X. Zheng, H. Han, S. Shi, Q. Fang, Z. Du, X. Hu, and Q. Guo, "InputSnatch: Stealing input in LLM services via timing side-channel attacks," 2024. [Online]. Available: https://arxiv.org/abs/2411.18191

[18] G. Wu, Z. Zhang, Y. Zhang, W. Wang, J. Niu, Y. Wu, and Y. Zhang, "I know what you asked: Prompt leakage via KV-cache sharing in multi-tenant LLM serving," in *Proceedings of the 2025 Network and Distributed System Security Symposium*. San Diego, CA, USA: Internet Society, 2025.

[19] C. Gu, X. L. Li, R. Kuditipudi, P. Liang, and T. Hashimoto, "Auditing prompt caching in language model APIs," 2025. [Online]. Available: https://arxiv.org/abs/2502.07776

[20] Z. Pang, W. Wang, and Y. Liao, "Cache partitioning for mitigating timing side-channel attacks in LLM serving systems," in *2024 6th International Conference on Frontier Technologies of Information and Computer*. Qingdao, China: IEEE, 2024, pp. 1238–1245.

[21] vLLM Team, "Cache isolation for security in vLLM prefix caching," 2025, documentation of request-level cache salts for prefix-cache isolation. [Online]. Available: https://docs.vllm.ai/en/latest/design/v1/prefix_caching.html

[22] K. Chu, Z. Lin, D. Xiang, Z. Shen, J. Su, C. Chu, Y. Yang, W. Zhang, W. Wu, and W. Zhang, "Selective KV-cache sharing to mitigate timing side-channels in LLM inference," 2025. [Online]. Available: https://arxiv.org/abs/2508.08438

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[24] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation*. Santa Clara, CA, USA: USENIX Association, 2024, pp. 193–210.

[25] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," *Proceedings of Machine Learning and Systems*, vol. 5, pp. 606–624, 2023.

[26] L. Wang, X. Zheng, X. Zhang, Y. Zhang, Y. Wu, and C. Wang, "Optileak: Efficient prompt reconstruction via reinforcement learning in multi-tenant llm services," *arXiv preprint arXiv:2602.20595*, 2026.

[27] R. Weiss, D. Ayzenshteyn, G. Amit, and Y. Mirsky, "What was your prompt? a remote keylogging attack on AI assistants," 2024. [Online]. Available: https://arxiv.org/abs/2403.09751

[28] N. Carlini and M. Nasr, "Remote timing attacks on efficient language model inference," 2024. [Online]. Available: https://arxiv.org/abs/2410.17175

[29] DeepSeek-AI, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[30] AI@Meta, "Llama 3 model card," 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[31] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.

[32] U.S. Census Bureau, "U.s. census given-name and surname lists," https://www.census.gov/topics/population/genealogy/data/2020_names.html, 2020, accessed: 2026-06-18.

[33] A. Ben Abacha and D. Demner-Fushman, "A question-entailment approach to question answering," *BMC Bioinform.*, vol. 20, no. 1, pp. 511:1–511:23, 2019. [Online]. Available: https://bmcbioinformatics.biomedcentral.com/articles/10.1186/s12859-019-3119-4

[34] G. Chua, "system-prompt-leakage dataset," https://huggingface.co/datasets/gabrielchua/system-prompt-leakage, accessed: 2026-06-18.

[35] R. Taori *et al.*, "Stanford alpaca: An instruction-following llama model," 2023. [Online]. Available: https://github.com/tatsu-lab/stanford_alpaca

APPENDIX A
DEFERRED PROOFS

This appendix collects the proofs of the equalization results of Section V; the statements, assumptions, and resulting parameter rules remain in the main text.

Proof of Lemma 1 (Trace reduction). Under (A2) the joint density of $\mathcal{T}$ given $B = b$ factorizes as $p_b(o)\,K(g \mid o)$ with the *same* kernel K. Writing $D(o) := p_{\mathsf{hit}}(o) - p_{\mathsf{miss}}(o)$ and using $K \geq 0$,

$$\mathrm{TV}\big(\mathcal{L}^{\mathcal{T}}_{\mathsf{hit}}, \mathcal{L}^{\mathcal{T}}_{\mathsf{miss}}\big) = \tfrac{1}{2} \iint \big|D(o)\big| K(g \mid o)\, dg\, do = \tfrac{1}{2} \int \big|D(o)\big|\, do, \tag{23}$$

since $\int K(g \mid o)\, dg = 1$; the last expression is $\mathrm{TV}(\mathcal{L}^{o}_{\mathsf{hit}}, \mathcal{L}^{o}_{\mathsf{miss}})$. The identity is exact: post-first-token emissions carry *no* additional information about B. Without (A2) only "$\geq$" holds, and the excess is exactly the residual that an ITL-aware distinguisher can mine. □

Proof of Lemma 2 (Bounded advantage and capacity). (a) For any rule ψ the advantage is at most

$$\sup_A \big|\mathcal{L}^{\mathcal{T}}_{\mathsf{hit}}(A) - \mathcal{L}^{\mathcal{T}}_{\mathsf{miss}}(A)\big| = \mathrm{TV}\big(\mathcal{L}^{\mathcal{T}}_{\mathsf{hit}}, \mathcal{L}^{\mathcal{T}}_{\mathsf{miss}}\big),$$

attained by the likelihood-ratio test (Le Cam); Lemma 1 and (A3) bound it by δ_{eq}.

(b) Let $\delta = \mathrm{TV}(\mathcal{L}^{o}_{\mathsf{hit}}, \mathcal{L}^{o}_{\mathsf{miss}}) \leq \delta_{\mathsf{eq}}$; the cases $\delta \in \{0, 1\}$ are immediate, so take $\delta \in (0, 1)$. With densities w.r.t. a dominating measure μ, set

$$R = \frac{\min(p_{\mathsf{hit}}, p_{\mathsf{miss}})}{1 - \delta}, \qquad P'_b = \frac{(p_b - p_{\bar{b}})_+}{\delta},$$

which are densities because $\int \min(p_{\mathsf{hit}}, p_{\mathsf{miss}})\, d\mu = 1 - \delta$ and $\int (p_b - p_{\bar{b}})_+ d\mu = \delta$, have disjoint supports, and give $p_b = (1 - \delta)R + \delta P'_b$ for both b. So we may sample equivalently: draw $Z \in \{c, d\}$ with $\Pr[Z = d] = \delta$ independent of B; if $Z = c$ draw $o_1 \sim R$, else $o_1 \sim P'_B$; then $g_{2:n} \sim K(\cdot \mid o_1)$ by (A2). Revealing Z only helps, so $I(B; \mathcal{T}) \leq I(B; \mathcal{T}, Z) = I(B; Z) + I(B; \mathcal{T} \mid Z)$. Here $I(B; Z) = 0$ since $Z \perp B$; given $Z = c$, $o_1 \sim R$ regardless of B and $g_{2:n}$ depends on B only through o_1, so $I(B; \mathcal{T} \mid Z = c) = 0$; given $Z = d$ bound trivially by $H(B) = h_2(\pi)$. Averaging, $I(B; \mathcal{T}) \leq \delta\, h_2(\pi) \leq \delta_{\mathsf{eq}}$, and $h_2 \leq 1$ gives $C_{\mathsf{eq}} \leq \delta_{\mathsf{eq}}$. □

Proof of Corollary 1 (Repeated measurement). Couple $\mathcal{L}^{\mathcal{T}}_{\mathsf{hit}}$ and $\mathcal{L}^{\mathcal{T}}_{\mathsf{miss}}$ maximally per coordinate: they agree with probability $1 - \delta_{\mathsf{eq}}$ each, hence in all R with probability $(1 - \delta_{\mathsf{eq}})^R$, and the TV distance between the product laws is at most the coupling-failure probability. For the mutual information, run R independent genies Z_i: $I(B; \mathcal{T}^{(R)}) \leq \Pr[\exists i : Z_i = d]\, h_2(\pi)$. □

Proof of Theorem 2 (Odometer–equalization composition). By the chain rule (13), $I(S^*; O_{1:k} \mid P) \leq \sum_{i=1}^{k} C_i$ with C_i the i-th query's capacity; the attacker fixes each continuation, so O_i depends on S^* only through that probe's bit B_i, and data processing gives $I(S^*; O_i \mid P, O_{<i}) \leq I(B_i; O_i \mid P, O_{<i}) \leq C_i$. A Phase-1 query is an unpadded oracle, so $C_i \leq C_1$ (the binary idealization gives $C_1 \leq 1$; as discussed after Theorem 1, a continuous TTFT can raise it to $C_1 = O(\log L)$, which replaces N_1 by $N_1 C_1$ below and only strengthens the case for a small N_1); a Phase-2 query has $C_i \leq \delta_{\mathsf{eq}}$ by Lemma 2(b), which requires the bound for *every* prior, not merely $\pi = \frac{1}{2}$. Hence $I(S^*; O_{1:k} \mid P) \leq N_1 + (k - N_1)\delta_{\mathsf{eq}}$, while Fano (12) gives $I \geq H(S^* \mid P) - h_2(\varepsilon) - \varepsilon \log_2(M - 1)$; solving for k yields (21). Each query's phase is measurable with respect to the attacker-controlled history, so N_1 may be taken at its worst case; novel branches contributed by other tenants only advance lockdown, never delay it. □

APPENDIX B
PERFORMANCE BOUNDS

The preceding results bound what an attacker can *learn*; we now bound what the defense *costs*, so that the security dial (q, η, G) comes with an explicit price tag. Three quantities matter: the latency of benign traffic, the worst-case delay of a padded request, and the serving capacity retained relative to isolation. Throughout, c_{ctrl} denotes the constant control-plane cost of one odometer decision, $\pi_2 \in [0, 1]$ the fraction of requests emitted in Phase 2, and $\tau_q(L) = (1 + \eta)\big(a + bL + (Q_q(R))_+\big)$ the clamp target of Equation 10 for a length-L request under lockdown.

Theorem 3 (Per-request latency bound)**.** *Under* Mirage*:* (a) *every Phase-0/1 request is emitted at* $e_1 - t_0 = t_{\mathsf{actual}} + c_{\mathsf{ctrl}}$*, i.e., benign and grace traffic is never padded and pays only the constant control-plane term;* (b) *every Phase-2 request satisfies*

$$e_1 - t_0 \;\leq\; \max\{t_{\mathsf{actual}},\, \tau_q(L)\} + \sigma + c_{\mathsf{ctrl}}, \tag{24}$$

so its added delay obeys $\Delta_{\mathsf{pad}} \leq (\tau_q(L) + \sigma - t_{\mathsf{actual}})_+$ *while every inter-token gap is preserved exactly;* (c) *averaging over the mix,* Mirage*'s expected TTFT inflation over No Defense is at most* $c_{\mathsf{ctrl}} + \pi_2\, \mathbb{E}\big[(\tau_q(L) + \sigma - t_{\mathsf{actual}})_+\big]$.

Proof. (a) holds by construction of the phase gate; (b) follows from Equations 6–10 and the jitter bound $\epsilon \leq \sigma$; (c) sums the two cases weighted by π_2. □

Isolation parity. Equation 24 yields a direct comparison point. A padded request is emitted no later than an η-margined upper quantile of a *genuine* cold miss, which is exactly the physical latency that isolation defenses impose on every cross-boundary request via recomputation. Mirage's worst per-request latency therefore exceeds isolation's typical one by at most the margin factor $(1 + \eta)$, the residual tail term $Q_q(R)$, and the jitter σ; unlike isolation, this worst case is charged only to the π_2 slice of traffic under active probing rather than unconditionally, and Appendix E shows the pessimal-accounting realization of this bound (87 vs. 88 ms).

Theorem 4 (Capacity dominance over isolation)**.** *Model each request as GPU demand* $w = d + p_{\mathsf{suf}}$*, where* d *is decode work and* p_{suf} *the prefill of the uncached suffix, and let the engine be work-conserving with maximal sustainable throughput* $\lambda_{\max} = \mu / \mathbb{E}[w]$ *for GPU capacity* μ*. Because* Mirage *executes padding*

as a gateway timer on already-computed chunks and metering on the host CPU, it adds zero GPU demand, hence

$$\lambda_{\max}^{\textit{Mirage}} = \lambda_{\max}^{\mathsf{NoDef}}, \qquad \frac{\lambda_{\max}^{\textit{Mirage}}}{\lambda_{\max}^{\mathsf{iso}}} = 1 + \frac{\bar{h}\,\mathbb{E}[p_P]}{\mathbb{E}[w]} \;\geq\; 1, \quad (25)$$

where p_P is the prefill cost of the shared prefix and $\bar{h}$ the fraction of requests whose cross-boundary reuse isolation is forced to drop.

Proof. Mirage leaves the engine's admission, scheduling, and kernels untouched, so per-request GPU demand equals No Defense's and the first equality follows from work conservation. Isolation converts each forbidden reuse into a recomputation, raising demand from w to $w + p_P$ on the $\bar{h}$ fraction, so $\mathbb{E}[w_{\mathsf{iso}}] = \mathbb{E}[w] + \bar{h}\,\mathbb{E}[p_P]$ and the ratio follows from $\lambda_{\max} = \mu/\mathbb{E}[w]$. □

Equation 25 makes the throughput gap of Figure 3 predictable rather than empirical: the observed $1.6\times$ headroom on Llama-3-8B corresponds to a workload where forbidden prefix reuse accounts for $\bar{h}\,\mathbb{E}[p_P]/\mathbb{E}[w] \approx 0.6$ of the undefended demand, and the gap widens monotonically with prefix length and hit rate, which is exactly what prefix caching is for.

Proposition 1 (The (q, η) dial: leakage vs. padding)**.** *Suppose the online estimate satisfies $\sup_x |\widehat{F}_{\mathsf{miss}}(x) - F_{\mathsf{miss}}(x)| \leq \varepsilon_{\mathsf{est}}$, and let $p_{\mathsf{slow}} = \Pr[t^{\mathsf{hit}}_{\mathsf{actual}} > \tau_q(L)]$ be the probability that a physical hit overshoots the clamp under load. Then the clamp emitter's equalization gap obeys*

$$\delta_{\mathsf{eq}} \;\leq\; (1-q) \;+\; \varepsilon_{\mathsf{est}} \;+\; p_{\mathsf{slow}}, \quad (26)$$

while the expected pad on a fast hit is $\mathbb{E}[\Delta_{\mathsf{pad}} \mid \mathsf{hit}] = \tau_q(L) + \sigma/2 - \mathbb{E}[t^{\mathsf{hit}}_{\mathsf{actual}}]$, which is nondecreasing in both q and η.

Proof. Condition on a Phase-2 prefix. A fast hit ($t \leq \tau_q$) is emitted at $\tau_q + \epsilon$; a miss is emitted at $\max(t, \tau_q) + \epsilon$, i.e., at $\tau_q + \epsilon$ with probability $F_{\mathsf{miss}}(\tau_q)$ and on its natural tail otherwise. The two emitted laws therefore coincide except on an event of probability at most $\big(1 - F_{\mathsf{miss}}(\tau_q)\big) + p_{\mathsf{slow}}$; since τ_q dominates the estimated q-quantile, $F_{\mathsf{miss}}(\tau_q) \geq q - \varepsilon_{\mathsf{est}}$, and a maximal coupling converts the event bound into the total-variation bound (26). The common jitter ϵ does not affect the distance. Monotonicity of the pad is immediate from the definition of τ_q. □

Proposition 1 closes the loop between the two analyses. Lemma 2 *assumed* an equalization gap δ_{eq} (A3); Equation 26 now bounds that gap by deployment-controlled quantities, and substituting it into Theorem 2 prices security in milliseconds: driving the per-probe capacity down to $(1-q) + \varepsilon_{\mathsf{est}} + p_{\mathsf{slow}}$ costs exactly the tail pad of Theorem 3(b), and both move monotonically with the same knob. The bound is also consistent with measurement: at q=0.97 it predicts $\delta_{\mathsf{eq}} \lesssim 0.03 + \varepsilon_{\mathsf{est}} + p_{\mathsf{slow}}$, in line with the measured $\widehat{\delta}_{\mathsf{eq}} \approx 0.06$ of Section VI-E, whose excess over 0.03 empirically attributes the residual to finite-window estimation and load-induced slow hits rather than to any structural leak of the emitter. Finally, the control-plane state is itself bounded: each metered prefix stores a decayed scalar and a branch-hash set, $O(1)$ plus $O(|B_P|)$ hashes, and B_P may be replaced by a fixed-size sketch whose one-sided error only *over*-counts novelty, a fail-safe direction that can tighten memory without weakening Equation 4.

Appendix C
Experimental Setup Details

Hardware and serving configuration. All experiments run on a multi-GPU server with NVIDIA A40 GPUs. The parallel runner assigns each SGLang victim server an exclusive GPU; detector, proposer, and proxy helper processes run on separate helper GPUs when available. The victim model is served with longest-prefix-match scheduling, a single in-flight request, and token-granularity paging. These settings intentionally sharpen prefix-cache timing and LPM order effects, making the configuration adversary-favorable for security testing.

Models and Datasets. We evaluate three models spanning three families and three parameter scales (1.8B, 8B, 14B): *DeepSeek-R1-Distill-Qwen-14B* [29], *Meta-Llama-3-8B-Instruct* [30], and *Qwen1.5-1.8B-Chat* [31].

Security experiments use a 500-sample corpus of confidential consultation prompts assembled entirely from public sources: patient names from U.S. Census given-name and surname lists [32], medical conditions and questions from MedQuAD [33] (`lavita/MedQuAD`), and a shared system prompt from `gabrielchua/system-prompt-leakage` [34]. Each prompt embeds a sensitive name and condition beneath a shared sensitive prefix. The overhead workload uses only public instruction-following prompts from `tatsu-lab/alpaca` [35]. Thus the overhead benchmark measures benign public-prefix reuse rather than mixed sensitive traffic.

Defense configuration. Mirage uses threshold $\theta = 1.0$, decay $\gamma = 0.995$/s, sensitivity weight $\omega = 1.0$ on sensitive prefixes, SGLang pagedattention block size $B = 1$ token, dual-track target margin 0.20 at the 0.97 quantile, and 10 ms uniform emission noise; this is the $G = 1$ grace budget, so the first novel branch beneath a sensitive prefix passes freely and a rapid second candidate triggers lockdown. The $B = 1$ setting is adversary-favorable and avoids relying on coarse physical KV blocks to suppress attacks.

Appendix D
Confidence Intervals

Every ASR reported in Table I is *directly measured* (Section VI-A); this appendix quantifies its sampling variability. EarlyBird and InputSnatch probe their finite candidate pools in full (EarlyBird's 467 dataset first-tokens, InputSnatch's full 500×500 name/condition dictionary), so ASR is measured at every budget. OptiLeak probes, for every sensitive token position, the $\leq$50-candidate rank window containing the ground truth at its true rank out to rank $10{,}000$; because the co-submitted batch stays ≤ 50 at any depth, the order channel is exercised directly at each budget without a rank-tail estimator.

Bootstrap protocol. We compute 95% percentile bootstrap intervals with $5{,}000$ resamples. EarlyBird and InputSnatch resample the 500 evaluation samples with replacement. OptiLeak resamples at the sample/prompt granularity and carries all token

positions from a sampled prompt together; this cluster bootstrap avoids treating tokens from one prompt as independent. An all-zero empirical channel has interval $[0, 0]$, which should be read as no observed channel in this experiment rather than a population-level impossibility proof; for measured-zero Mirage cells we report a rule-of-three upper bound instead.

TABLE VI
HEADLINE ASR WITH 95% PERCENTILE BOOTSTRAP CIS (%): EARLYBIRD AND OPTILEAK AT ASR@10K, INPUTSNATCH END-TO-END. MIRAGE MEASURED-ZERO CELLS SHOW A RULE-OF-THREE UPPER BOUND.

Model	Attack	No Defense	Mirage (Ours)
DeepSeek-14B	EarlyBird	92.4 [90.0, 94.6]	0.2 [0.0, 0.6]
	InputSnatch	59.4 [55.0, 63.8]	0.4 [0.0, 1.0]
	OptiLeak	68.8 [66.0, 71.6]	0.0 [0.0, 0.1]
Llama-3-8B	EarlyBird	33.4 [29.4, 37.6]	0.0 [0.0, 0.6]
	InputSnatch	34.2 [30.0, 38.4]	0.4 [0.0, 1.0]
	OptiLeak	81.8 [79.1, 84.3]	0.0 [0.0, 0.1]
Qwen1.5-1.8B	EarlyBird	6.2 [4.2, 8.4]	0.0 [0.0, 0.6]
	InputSnatch	0.0 [0.0, 0.0]	0.0 [0.0, 0.6]
	OptiLeak	67.6 [65.6, 69.6]	0.0 [0.0, 0.1]

APPENDIX E
BENIGN SHARING COST

What happens when a sensitive prefix is never attacked but legitimately shared by many users? To answer this, we stress-test the most pessimistic configuration on Llama-3-8B: a single global budget that does not distinguish between individual users. Under this setting, a sustained stream of distinct benign queries from different users looks mathematically identical to an attacker's fan-out probing. Consequently, the mechanism errs on the side of caution and triggers lockdown, padding 93% of these legitimate requests. While this high padding rate represents a theoretical drawback of a purely global budget, it is fundamentally a deliberate fail-safe behavior: when the system cannot separate identities, it prioritizes privacy.

More importantly, the actual penalty for this "false positive" is remarkably cheap. Because Mirage applies the delay strictly at the gateway, the padded request is still served from the shared GPU cache. The worst-case penalty is merely a bounded ≈ 60 ms shift (87 ms padded versus the native 25 ms hit). In other words, Mirage's absolute worst-case benign penalty costs exactly what isolation defenses charge unconditionally on every request. Furthermore, this global budget is merely a diagnostic floor. As Table VII demonstrates, enabling trust-scoped metering completely resolves this flaw. By assigning each authenticated user their own budget, the mechanism cleanly separates normal usage from malicious fan-out, immediately dropping the benign padding rate to 0% and restoring native latency, while remaining fully capable of locking down cross-account Sybil attacks.

APPENDIX F
SENSITIVE-SHARED WORKLOADS

This dynamic padding behavior directly translates to massive throughput advantages over selective isolation systems like SafeKV. We demonstrate this by comparing 64 benign tenants sharing a single sensitive prefix under varying loads. On purely public traffic, all systems perform identically (87–89 ms). On sensitive traffic at a low load of 8 req/s, the systems also appear matched: Cache Partition and SafeKV sit at 88 ms, while Mirage records 91 ms. This tight alignment highlights the intelligence of Mirage's dynamic estimator. Although Mirage actively pads these requests under the global budget stress test, the estimator targets the *actual* concurrent miss latency. Because cache misses are relatively fast at low loads, Mirage's padded hit latency perfectly matches the physical recomputation latency of isolation defenses, so it effectively masks the padding penalty it applies.

TABLE VII
BENIGN-SHARING STRESS TEST (LLAMA-3-8B, G=1, PESSIMAL GLOBAL BUDGET): A PAD IS A SMALL SHIFT ON A STILL-CACHED HIT; SCOPED METERING REMOVES IT. †SIMULATION.

Metering policy	Benign pattern	Padded	TTFT (ms)
Global budget	distinct continuations	93%	87
Global budget	exact repeats (ρ=0)	**0%**	**25**
Trust-scoped†	distinct continuations	**0%**	**25**

However, the mechanisms diverge drastically as the load approaches the server's capacity at 16 req/s. Strict isolation forces the GPU to redundantly recompute the sensitive prefix for every single tenant. This redundant prefill quickly saturates the GPU compute capacity, inflating TTFT to nearly 1.8 s (a severe $3\times$ degradation). In stark contrast, Mirage continues to reuse the shared cache at full GPU speed, merely applying its gateway-level padding constraint. This allows Mirage to maintain a highly responsive 611 ms TTFT, closely tracking the undefended 542 ms baseline. Ultimately, while Mirage introduces a minor gateway delay on sensitive hits to preserve security, this deliberate trade-off completely neutralizes the catastrophic GPU queuing collapse inherent to isolation defenses when throughput is contended.

TABLE VIII
TTFT (MS), 64 TENANTS SHARING ONE SENSITIVE PREFIX (LLAMA-3-8B, MRR=16): NEAR CAPACITY, RECOMPUTATION INFLATES ISOLATION $\sim 3\times$; MIRAGE TRACKS NO DEFENSE.

Defense	Public	Sensitive-shared	
	@8 req/s	@8 req/s	@16 req/s
No Defense	87	86	542
Mirage	89	91	611
Cache Partition	88	88	1858
SafeKV (selective)	89	88	1777

APPENDIX G
SECURITY UNDER REAL BATCHED LOAD

Our main security numbers use a single in-flight request (MRR=1), the adversary-favorable setting that sharpens the timing signal, whereas a production server batches. We therefore re-run both channels against Mirage at MRR=16 with concurrent benign background load. The order channel stays at the guessing floor (0% channel success over 120 targets): probes that land in the same batch are released through the same phase-gated barrier and fresh random permutation, and an attacker who forces serialization by occupying the run slots with slow filler merely recovers the MRR=1 regime that the batch-order equalization already closes. The magnitude/ITL channel likewise stays at chance: the whole-trace distinguisher

TABLE IX
GRACE-BUDGET TRADE-OFF (QWEN1.5-1.8B, 5 SEEDS): BENIGN HIT LATENCY SATURATES AT G=1; SPREAD-CANDIDATE ASR STAYS AT THE 12.5% BASELINE FOR EVERY G.

Grace budget	G=0 (pad-all)	**G=1** (default)	G=2	G=3
Benign hit latency (ms) ↓	476	**44**	42	46
Spread-candidate ASR (%)	11.6	12.0	12.1	11.1

of Table IV stays within 0.53–0.58 effective AUC against Mirage under MRR=16. This holds because the clamp targets an *upper* quantile of the *current* miss window, which only rises as batching thickens the miss tail, so added load can push the emitter toward over-padding but never below the concurrent miss law. Both the MRR=1 and the MRR=16 batched regimes are thus covered by our evaluation.

APPENDIX H
PARAMETER SWEEPS AND THE LOW-AND-SLOW ADVERSARY

This appendix collects the scalar parameter studies that fix Mirage's defaults and the analysis of the low-and-slow adaptive adversary that the grace budget admits (Section VI-E). Each sweep varies one knob with the rest held at the main configuration.

Grace budget G. G trades benign latency against the size of the unpadded window, and we justify the default G=1 by measuring *both* axes on one model (Qwen1.5-1.8B; Table IX). *Latency:* G=0 pads every benign cache hit up to the miss target (476 ms), whereas any $G \geq 1$ serves it at the true hit latency ($\approx$ 44 ms), a $>10\times$ reduction that *saturates at* G=1, since G=2 and G=3 measure the same $\approx$ 44 ms. *Security:* the realistic spread-candidate top-1 ASR (Section VI-E, no per-candidate oracle reset, 8 candidates so a 12.5% random baseline) stays at that baseline for every G (averaged over 5 seeds), so the wider window carries no realizable leakage. The trade-off is thus one-sided past G=1: the full latency benefit is already captured at G=1, while the unpadded window, and hence the theoretical exposure, grows linearly with G. The principled choice is the smallest budget that saturates the benefit, G=1: it avoids G=0's $>10\times$ benign-latency penalty and takes on none of the enlarged window of $G \geq 2$. We retain G=0 as an option for providers who want to pad even first accesses (the only setting that also closes the stacked-oracle bound of Table XII).

Repeat weight ρ. We ablate ρ with the exact-repeat evasion: an attacker seeds a candidate, waits for the score to decay, and re-issues the *identical* prompt hoping for an unpadded hit (200 trials, R=5; Table X). Realistically this already fails (effective AUC 0.50) through *self-cache pollution*, the seed caches the attacker's own wrong candidate and destroys the hit/miss contrast. Even under an idealized oracle that removes that pollution and keeps the victim's secret resident, which does reopen the channel (0.975), setting ρ=1 to charge exact repeats forces the harvest probe back into lockdown and closes it (0.52). We default to ρ=0 (free retries for benign duplicate traffic) because the oracle is not realizable under cache eviction (Section VI-E), and expose ρ=1 for providers unwilling to rely on residency.

TABLE X
REPEAT-WEIGHT ABLATION (EXACT-REPEAT EVASION, EFFECTIVE AUC). SELF-POLLUTION ALREADY NEUTRALIZES IT; THE IDEALIZED ORACLE IS CLOSED BY CHARGING REPEATS (ρ=1).

Setting	ρ	**Eff. AUC →.5**
Realistic (self-pollution)	0	**0.501**
Idealized oracle	0	0.975
Idealized oracle, charged	1	**0.516**

TABLE XI
CLAMP QUANTILE q AND MARGIN η SWEEP (QWEN1.5-1.8B): THE RESIDUAL STAYS NEAR CHANCE WHILE THE PAD MOVES MODESTLY, WITH NO KNIFE-EDGE.

Config	**Eff. AUC →.5**	**Mean pad (ms) ↓**
q=0.97, η=0.20 (default)	0.58	28
q=0.90, η=0.20	0.52	27
q=0.97, η=0.10	0.54	24
q=0.97, η=0.30	0.51	33

Emitter quantile q and margin η. The clamp target is set by the miss quantile q and the safety margin η (Section IV-E); to check that the default q=0.97, η=0.20 is not a delicate operating point, we sweep both and measure the residual channel (the 8-feature distinguisher's effective AUC) and the mean pad (Table XI). Across $q \in \{0.90, 0.97\}$ and $\eta \in \{0.10, 0.20, 0.30\}$ the residual stays near chance (0.51–0.58) while the pad rises gently and monotonically with the margin (24 → 33 ms). Security is thus not a knife-edge in these knobs: within this range the channel stays closed and only the overhead moves, so q=0.97, η=0.20 is a reasonable operating point rather than a tuned one. The trend is the expected one: more padding buys marginally more robustness at a small, predictable latency cost, and a provider can slide along it without reopening the channel.

Low-and-slow time-decay evasion. The second strategy waits for the odometer score to decay before probing. Re-issuing the *exact* same candidate is self-defeating and, even under an oracle, is closed by the repeat weight ρ; we treat this as the ρ ablation (Section VI-F). The more dangerous variant spreads *distinct* candidates over time, which we analyze here.

Spread-Candidate: the attacker avoids self-pollution by spacing *distinct* candidates over time, waiting for the score to decay below θ between probes so each is served in Phase 1. Our G=1 default admits one such unpadded probe per decay window. We report its worst case in Table XII, and we stress that the 76.5% figure (ρ=0; 25.5% at ρ=1) is an artifact of three oracle assumptions stacked together, not a channel that exists in deployment. Concretely, it requires simultaneously that (a) the odometer score is reset to exactly zero before *every* candidate, which forces the attacker to idle out a full multi-minute decay per probe (Equation 20); (b) throughout those cumulative hours the victim's *exact* secret prefix stays resident in the shared cache; and (c) the attacker's own probes never pollute that state. Assumptions (a) and (b) are directly contradictory under real load: the same idle time that decays the score is far more than enough for competing multi-tenant traffic to LRU-evict the victim's prefix, at which point every delayed probe is a natural

miss and recovers nothing. This is exactly the spatio-temporal dilemma of Equation 19, and a provider makes it explicit by choosing γ so the per-probe wait exceeds the cache residency time $\tau_{\max}$. Even setting residency aside, the knobs Mirage already exposes each close the window independently: charging exact repeats (ρ=1) drops the oracle ASR to 25.5%, and G=0 pads the first probe and eliminates the unpadded window by construction. We therefore treat the spread-candidate rows as a labeled upper bound that quantifies the cost of the grace budget, not as a realizable attack.

TABLE XII
WHITE-BOX ADAPTIVE ATTACKS (QWEN1.5-1.8B). THE DISTINGUISHER STAYS AT CHANCE AGAINST THE CLAMP; SPREAD-CANDIDATE ROWS ARE A STACKED-ORACLE BOUND, CLOSED BY ρ=1 OR G=0.

Configuration	R	Eff. AUC →.5	Top-1 ASR ↓
Multivariate Distinguisher			
No Defense	10	0.807	–
No Defense	20	0.962	–
Mirage (clamp)	20	**0.53 ±.01**	–
Spread-Candidate Scan (R=5, baseline ≈ 0.51%)			
Oracle ($\theta = 1.0, \rho = 0$)	5	–	76.5%
Oracle ($\theta = 1.0, \rho = 1$)	5	–	25.5%

APPENDIX I
LIMITATIONS

Mirage makes deliberate trade-offs, and we state its boundaries plainly.

Sensitivity Classification. Lemma 2 relies on correctly identifying sensitive prefixes. While our evaluation uses a lexical detector for benchmark reproducibility, Mirage's architecture does not inherently depend on text scanning. In real-world deployments, providers can adopt lightweight embedding-based sensitivity token detectors, or a fail-closed policy using existing application-level metadata (e.g., enterprise tenant IDs, private document flags, or proprietary endpoints) to route requests to the protected path ($\omega_{\mathsf{sens}} > 0$) by default. Under this paradigm, content scanning serves merely as an optional performance whitelist to safely bypass the odometer for known-public templates, completely eliminating the risk of missing obfuscated, paraphrased, or multilingual secrets.

Benign sharing of a metered prefix. Under the pessimal single-global-budget accounting, a sustained stream of distinct benign continuations beneath one fail-closed prefix is mostly padded (Appendix E). The pad is a bounded gateway shift on a request still served from the shared cache (no recomputation, no throughput loss), so this worst case matches what isolation charges every request unconditionally; exact repeats are never charged, trust-scoped accounting removes the padding for authenticated traffic entirely, and G is a tunable dial.

Estimator calibration. The miss target τ is tracked online; during cold-start warmup Mirage falls back to conservative default miss timing rather than leaving Phase 2 unprotected. The threat model allows dummy traffic and cache flushes (Section III), which shift the miss distribution the estimator tracks, but the estimator's structure leaves such manipulation little to work with. Shifting the distribution *up* only raises τ: more padding, never less. Shifting it *down* is structurally impeded three times over. The miss track admits only zero-cached-token observations, whose latency is bounded below by a genuine full prefill (partially cached "warm" requests are routed to the hit track and cannot dilute the miss window), and the affine fit normalizes away length games such as short-prompt floods. The target couples a margined affine center with the upper quantile of its recent residuals, so depressing it requires dominating essentially the entire recent pure-miss window, and the slow EMA tracks (β=0.9995) that seed and backstop the fit resist rapid displacement. And the genuine concurrent misses against which a padded hit would be compared are drawn from the same load regime the attacker's own flooding creates, so the reference moves with the window; the non-stationary MRR=16 runs of Appendix G exercise exactly this shifting-mix regime without reopening the channel. Hardening the window further with robust statistics is straightforward if a deployment wants defense in depth.

Distributed deployment. Our prototype keeps the odometer in a single tokenizer manager. A multi-replica deployment shares this state so that probes spread across replicas draw on one budget; because the state is small and additive (a decayed score and a branch-hash set per prefix), it maps directly onto the shared low-latency stores or per-prefix sticky routing that multi-replica gateways already employ for rate limiting. Quantifying the cross-replica read cost on the critical path is routine engineering we have not yet performed.

Hardware and scale. Our evaluation runs on NVIDIA A40 GPUs, single-GPU serving, and models up to 14B. We do not evaluate A100/H100 hardware, tensor- or pipeline-parallel multi-GPU serving, or 70B/MoE models, where prefill/decode timing, batch composition, and cache pressure differ. Because Mirage lives entirely in the gateway/control plane and adds no GPU work, we expect the mechanism to carry over, but the specific overhead and $\widehat{\delta}_{\mathsf{eq}}$ numbers should be read as characteristic of this regime, not of every stack out there.

Semantic-cache generalization. Section VI-D shows the *principle* transfers to embedding-similarity caches; it is a generalization of the KV-cache mechanism, not a full port of the KV-cache attacks. Our evaluation there uses a 40-topic medical corpus with real sentence embeddings and simulated hit/miss timing; real (rather than transplanted) semantic-cache attacks against a production embedding cache are still open for future work.

Threat scope. Mirage is purpose-built to neutralize API-visible timing side-channels arising from prefix-cache reuse, which currently represent the most accessible and scalable vector for cross-tenant prompt recovery. Consequently, it is not designed to be a panacea for all prompt recovery vulnerabilities. Specifically, Mirage does not address non-timing side-channels, such as analyzing encrypted network traffic bursts, monitoring token-length emission sequences or direct KV cache inversion attacks that extract tokens from raw KV cache tensors.

APPENDIX J
FUTURE WORK

Several directions follow directly from the limitations above.

Robust sensitivity gating. Replacing the lexical detector with a learned or entropy-based gate that scores a prefix by the conditional entropy and PII-likelihood of its continuations, rather than by membership in a word list, would extend metering to paraphrased, obfuscated, and multilingual secrets and remove the benchmark-overfitting concern.

Entropy-adaptive grace budget. Equation 16 ties the safe grace budget to the secret's min-entropy, which suggests granting more grace (and thus lower overhead) to provably hard-to-guess prefixes by estimating $H_\infty(S^* \mid P)$ online. The obstacle is that the serving model's next-token distribution measures linguistic likelihood, not sensitivity, and cannot lower-bound a stronger fine-tuned adversary's prior, so a naive estimate can grant unsafe grace. A safe version would use a conservative lower-confidence estimate and cap the result at the global budget G, $G_P = \min(G, \lfloor \alpha\, 2^{\hat{H}_\infty^{\mathrm{LCB}}} \rfloor)$, so estimation error can only tighten protection; we leave a careful instantiation to future work.

Extending threat coverage. Addressing non-timing leakage channels, such as token counts, network traffic, or direct KV cache inversion, would broaden the protection scope of Mirage. Techniques like differential privacy, traffic shaping, or secure multi-party computation could be explored to mitigate these additional non-timing threats in future systems.